

硕 士 学 位 论 文

基于大语言模型的知识图谱问答技术研究

A Research on LLM-based Technology for Knowledge Graph
Question Answering

作 者 姓 名: 王梦涵

学 号: 32217018

指 导 教 师: 赵哲焕

企 业 导 师: 祁贺

学 科、专 业: 软件工程

答 辩 日 期: 2025 年 5 月

大连理工大学

Dalian University of Technology

硕士学位论文答辩委员会

答辩人：王梦涵

答辩委员会委员：

大连理工大学教授： 郭成（主席）

大连理工大学副教授： 林驰、王洁、王胜法

大连钜智信息科技有限公司高工： 张元烨

答辩日期：2025 年 05 月 19 日

答辩地点：大连理工大学开发区校区综合楼 504 室

摘 要

随着人工智能技术的快速发展,知识图谱问答(KGQA)任务成为自然语言处理领域的研究热点,同时也面临诸多挑战。当前 KGQA 的难点主要体现在正确回答复杂多跳问题。而大语言模型(LLMs)强大的语言理解与生成能力给该研究带来了新的思路。

当前,利用大语言模型进行 KGQA 任务主要采用两种策略:一是基于知识图谱推理路径规划的策略,即通过 LLMs 构建从问题到答案的多跳推理路径,利用知识图谱中的结构信息逐步推导出最终答案;二是基于知识图谱信息检索的策略,即通过 LLMs 检索与问题相关的实体和关系信息,并整合推理生成准确答案。针对 KGQA 中复杂多跳问题中的关键挑战,本文围绕上述两种策略展开深入探索。

基于推理路径规划策略,针对 KGQA 复杂多跳问题中的推理链解析不准确的问题,本文提出了基于自我验证增强的图谱问答智能体。该方法旨在利用大模型生成高效的推理路径。通过设计问答智能体和验证智能体构建多轮迭代验证机制,优化推理路径和答案生成。Self-VEQA 还引入记忆机制积累历史问答经验,提升模型在类似问题上的表现。

基于知识图谱信息检索策略,针对 KGQA 复杂多跳问题中因问题复杂性导致相关知识检索不全面的挑战,本文提出了基于迭代多跳知识检索与推理的图谱问答模型。该方法旨在利用大模型高效地检索知识图谱和推理。该方法通过问题分解策略将复杂多跳问题拆解为单跳子问题,降低了检索和推理的难度;结合迭代的知识图谱检索机制,逐步定位相关三元组,并提出知识整合技术,将子问题答案与当前问题融合,构建连贯的上下文信息,从而提升三元组检索的准确性。

关键词: 知识图谱问答; 大语言模型; 智能体; 知识检索增强

ABSTRACT

With the rapid advancement of artificial intelligence technology, Knowledge Graph Question Answering (KGQA) has become a research hotspot in the field of natural language processing, while also facing numerous challenges. The current difficulties in KGQA are mainly reflected in correctly answering complex multi-hop questions. The powerful language understanding and generation capabilities of large language models (LLMs) have brought new perspectives to this research.

Currently, there are two main strategies for utilizing large language models in KGQA tasks: one is the knowledge graph reasoning path planning strategy, which involves constructing multi-hop reasoning paths from questions to answers using LLMs, leveraging the structural information in the knowledge graph to gradually derive the final answer; the other is the knowledge graph information retrieval strategy, which retrieves entity and relation information related to the question through LLMs and integrates reasoning to generate accurate answers. Addressing the key challenges in complex multi-hop questions in KGQA, this paper delves into the exploration of the aforementioned two strategies.

Based on the reasoning path planning strategy, to tackle the issue of inaccurate reasoning chain parsing in complex multi-hop KGQA questions, this paper proposes a self-verification enhanced question answering agent. This method aims to utilize large models to generate efficient reasoning paths. By designing a question answering agent and a verification agent to construct a multi-round iterative verification mechanism, it optimizes reasoning paths and answer generation. Self-VEQA also introduces a memory mechanism to accumulate historical question answering experiences, enhancing the model's performance on similar questions.

Based on the knowledge graph information retrieval strategy, to address the challenge of incomplete knowledge retrieval due to the complexity of questions in complex multi-hop KGQA, this paper proposes an iterative multi-hop knowledge retrieval and reasoning framework. This method aims to efficiently retrieve knowledge graphs and reason using large models. By employing a question decomposition strategy to break down complex multi-hop questions into single-hop sub-questions, it reduces the difficulty of retrieval and reasoning; combined with an iterative knowledge graph retrieval mechanism, it gradually locates relevant triples and proposes knowledge integration technology to merge sub-question answers with the current question, constructing coherent contextual information, thereby improving the accuracy of triple retrieval.

Key Words: KGQA; LLM; Agent; RAG

目 录

1 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外相关研究工作进展	2
1.2.1 传统的知识问答方法.....	2
1.2.2 基于大语言模型的推理.....	3
1.2.3 基于大语言模型的智能体.....	4
1.2.4 基于大语言模型的知识图谱问答.....	4
1.3 研究内容和主要工作.....	5
1.4 论文主要结构.....	6
2 相关理论与技术	7
2.1 基本定义.....	7
2.1.1 知识图谱	7
2.1.2 知识问答	7
2.2 相关技术.....	8
2.2.1 大语言模型.....	8
2.2.2 检索增强生成.....	11
2.2.3 提示工程.....	12
3 基于自我验证增强的图谱问答智能体	14
3.1 引言.....	14
3.2 方法.....	14
3.2.1 模型概述.....	14
3.2.2 跳数预测模块.....	16
3.2.3 问答模块.....	17
3.2.4 验证模块.....	19
3.2.5 记忆模块.....	20
3.3 实验.....	21
3.3.1 数据集.....	21
3.3.2 评估指标.....	22
3.3.3 基线模型.....	23
3.3.4 实现细节	23
3.3.5 实验结果.....	23
3.3.6 消融实验.....	26
3.3.7 其他分析.....	27
3.4 本章小结.....	30

4 基于迭代多跳知识检索与推理的图谱问答模型	31
4.1 引言	31
4.2 方法	32
4.2.1 模型概述	32
4.2.2 语义扩充	32
4.2.3 问题分解	33
4.2.4 知识图谱检索	35
4.2.5 知识整合	36
4.2.6 大模型推理	37
4.2.7 案例分析	38
4.3 实验	39
4.3.1 数据集	39
4.3.2 评估指标	39
4.3.3 基线模型	39
4.3.4 实现细节	40
4.3.5 主要结果与分析	40
4.3.6 消融实验	43
4.3.7 其他分析	44
4.4 本章小结	46
5 结论与展望	48
5.1 结论	48
5.2 展望	48
参考文献	49

图目录

图 2.1 知识图谱	7
图 2.2 知识问答示例	8
图 2.3 Transformer 架构图	10
图 2.4 LLMs 直接回答问题和使用知识图谱问答	11
图 3.1 Self-VEQA Agent 框架图	15
图 3.2 Self-VEQA Agent 算法流程	16
图 3.3 问答智能体 prompt	17
图 3.4 完整的推理链	18
图 3.5 验证模块 prompt	20
图 3.6 Self-VEQA Agent 案例分析	21
图 3.7 迭代轮数对性能的影响(WebQSP)	28
图 3.8 推理链的数量对性能的影响(WebQSP)	30
图 4.1 Iter-MKGRR 的整体框架	32
图 4.2 Iter-MKGRR 算法流程	33
图 4.3 问题分解的 prompt	34
图 4.4 知识检索详细过程	35
图 4.5 知识整合部分的 prompt	36
图 4.6 案例分析	38

表目录

表 3.1 Hit@1 结果(MetaQA)	24
表 3.2 Hit@1 和 F1 结果(WebQSP)	25
表 3.3 消融实验(MetaQA)	26
表 3.4 消融实验(WebQSP)	27
表 3.5 交互轮数对性能的影响(WebQSP)	27
表 3.6 推理链的数量对性能的影响(WebQSP)	29
表 4.1 Hit@1 结果(MetaQA)	40
表 4.2 F1 和 Hit@1 结果(WebQSP)	41
表 4.3 消融实验(MetaQA)	44
表 4.4 消融实验(WebQSP)	44
表 4.5 语义扩充数量对 Hit@1 的影响(MetaQA)	45
表 4.6 语义扩充数量对性能的影响(WebQSP)	45
表 4.7 推理大模型对 Hit@1 的影响(MetaQA)	46
表 4.8 推理大模型对 Hit@1 和 F1 的影响(WebQSP)	46

1 绪论

1.1 研究背景与意义

随着人工智能技术的快速发展,知识图谱(Knowledge Graph, KG)作为一种结构化的知识表示方式,为机器理解人类知识提供了重要基础。在这一背景下,知识图谱问答(Knowledge Graph Question Answering, KGQA)作为自然语言处理(Natural Language Processing, NLP)领域的一个重要研究方向,受到了广泛关注。KGQA^[53]的核心任务是将自然语言问题映射为知识图谱的结构化查询,并根据查询的结果回答问题。KGQA 的重要性在于其能够将人类的自然语言^[10]与机器可理解的结构化知识无缝连接,从而支持智能问答、智能客服、医疗诊断、金融分析、教育辅助等多个关键应用场景。例如,在医疗领域,KGQA 可以帮助医生快速检索疾病的相关治疗方案;在教育领域,它可以为学生提供精准的知识点解答;在金融领域,它能够辅助分析师高效获取市场数据。这些应用不仅提升了信息检索的效率和准确性,还推动了社会智能化进程,为各行各业的数字化转型提供了重要技术支持。

尽管近年来深度学习、图神经网络(Graph Neural Networks, GNN)等技术的进步推动了 KGQA 的发展^[44],但处理复杂问题,尤其是多跳推理^[49]问题,仍然是 KGQA 领域的主要挑战。多跳推理^[54]要求模型能够通过多个步骤从知识图谱中提取信息,这对语义理解、知识检索和推理能力提出了更高的要求。例如,回答“某位科学家的导师的母校在哪里?”这类问题,需要在知识图谱中依次找到科学家、导师及其母校,并进行多步推理。这种复杂问题的解决能够显著提升 KGQA 任务的实用性。

近年来,大语言模型^[64-65](Large Language Models, LLMs)的兴起为 NLP 领域注入了新的活力,也为 KGQA 带来了新的研究思路。凭借其强大的语言理解和生成能力,LLMs^[35]在文本生成、机器翻译等任务中展现出卓越的性能,同时也为 KGQA 中的复杂多跳推理问题提供了新的解决途径。然而,尽管 LLMs 在 KGQA 任务中展现出巨大潜力,其在处理复杂多跳问题时仍存在一定的局限性。

因此,针对 LLMs 在复杂多跳问题上的局限性,本文分别提出了两种解决方案:一是基于自我验证增强的知识图谱问答智能体(Self-Verification Enhanced Question Answering Agent, Self-VEQA Agent),通过多轮迭代验证机制优化推理路径和答案生成;二是基于知识图谱信息检索的迭代多跳知识检索与推理模型(Iterative Multi-hop Knowledge Retrieval and Reasoning, Iter-MKGRR),通过问题分解和迭代检索机制提升知识检索的全面性和准确性。这些方法旨在解决 LLMs 在复杂多跳推理中的不足,进一步

提升 KGQA 任务的性能和应用价值。

1.2 国内外相关研究工作进展

1.2.1 传统的知识问答方法

基于检索的知识图谱问答(Retrieval-based Knowledge Question Answering)^[57] 是一种通过从知识图谱中检索相关信息来回答用户问题的方法。其核心思想是通过检索知识图谱中的结构化信息，直接或间接地找到与用户问题相关的答案^[68]。该方法通常包括问题解析、知识图谱检索和答案生成。其中问题解析依赖于自然语言处理(NLP)技术，如分词、词性标注和命名实体识别，以提取问题的关键语义信息^[8]；知识图谱检索通过语义匹配或向量化表示技术找到与问题最相关的知识；最后，基于检索结果生成答案。为了提高问答的可解释性和推理能力，研究者提出了多种改进方法，其中 KV-mem 方法^[1]是一个典型代表。

KV-mem 方法是基于键值记忆神经网络(Key-Value Memory Networks)的一种改进方法，它通过将答案-问题对视为键值对来实现可解释的推理。具体而言，键(Key)表示问题或问题的某种语义表示，值(Value)表示对应的答案。在训练过程中，模型学习将问题映射到记忆单元中的键值对；在推理阶段，模型通过检索与当前问题最相关的键值对，并组合多个键值对中的信息生成答案。例如，当用户提出一个复杂问题时，KV-mem 方法能够显式地检索知识库中与之相关的多个键值对，并通过逻辑组合生成最终答案。这种设计不仅提高了模型的推理能力，还增强了其可解释性。

基于嵌入的方法^[43-45]在知识图谱问答^[2](KGQA)中利用知识图谱嵌入(Knowledge Graph Embedding, KGE)技术^[47]，将知识图谱中的实体和关系映射到低维向量空间^[58]。这种方法能够捕捉实体和关系之间的语义信息，并通过向量运算来进行推理和问答。基于嵌入的方法^[42]通常通过学习知识图谱的向量表示和问题的嵌入表示，将两者结合以预测答案。尽管这些方法在 KGQA 任务中表现出色，但它们也存在一些局限性，例如训练过程耗费大量时间和资源、对数据集质量要求较高，以及模型泛化能力不足等问题。

EmbedKGQA^[1]是一种典型的基于嵌入的方法，它利用知识图谱嵌入技术来解答多跳自然语言问题。具体而言，EmbedKGQA 首先在嵌入空间中学习知识图谱的表示，将实体和关系映射到低维向量空间；在此基础上，针对给定的问题，学习其问题嵌入；最后，通过结合知识图谱嵌入和问题嵌入来预测答案。这种方法能够捕捉实体和关系之间的语义信息，从而支持复杂的多跳推理。然而，EmbedKGQA 的训练过程需要大量的计算资源和时间，尤其是在处理大规模知识图谱时，其效率受到限制。

UniKGQA^[32]是另一种基于嵌入的方法，其利用预训练语言模型(PLM)，把检索、推

理与语义匹配模块整合起来，用于实现问题-关系的语义匹配。具体来说，UniKGQA 通过预训练语言模型，如 BERT、GPT 等，来编码问题和知识图谱中的实体和关系，然后通过联合训练的方式，使得问题和答案在向量空间中更加接近。这种方法能够有效地利用预训练模型的强大表示能力，提高问答系统的性能。

尽管上述方法在知识图谱问答(KGQA)任务中提升了性能，但它们在模型训练时存在一些明显的不足。一方面，训练需要耗费大量的时间和资源；另一方面，对数据集有着较为严格的要求。而且，训练得到的模型通常缺乏泛化能力。具体来说，传统方法通常需要大量的计算资源和时间来进行训练，尤其是在处理大规模知识图谱时。例如，EmbedKGQA 需要在嵌入空间中学习知识图谱的表示，这需要大量的计算资源和时间。对于数据集，这些方法通常依赖于高质量、大规模的训练数据集。如果数据集不够丰富或存在噪声，模型的性能可能会受到显著影响。对于泛化能力，传统方法在训练过程中通常依赖于特定的数据集和任务，导致模型在面对新的、未见过的数据或任务时，表现不佳。例如，KV-mem 方法在处理复杂问题时表现良好，但在面对新的问题类型时，可能无法有效泛化。

1.2.2 基于大语言模型的推理

在当今的人工智能领域，基于大语言模型的推理^[69]技术广受关注。大语言模型，如 ChatGPT 系列等，凭借其海量的参数与对海量文本数据的学习，具备了强大的语言理解与生成能力。从技术原理层面剖析，在推理过程中，模型依据输入文本的特征，运用预训练阶段所习得的知识与模式，通过复杂的神经网络架构进行信息处理。当面对诸如问答任务^[28]时，模型会对问题进行语义解析，从自身庞大的知识储备里检索关联信息，并遵循一定逻辑规则组织答案，以此展现推理能力。

大语言模型(LLM)最初是为文本生成而设计的，但当把它应用到自然语言处理的其他子领域时，却展现出了卓越的性能^[7]。尤其是大语言模型的推理能力，在人工智能研究领域引起了广泛的关注^[12]。一些研究对大语言模型在多种推理方面的表现进行了探索，涵盖了算术推理、逻辑推理以及常识推理等。这些出色的表现，让大语言模型成为了其他领域任务中理想的推理工具。

基于大语言模型的推理，多数为基于提示工程的^[5]方法。从本质上讲，基于提示的大语言模型推理依赖于精心设计的提示信息。这些提示作为学习的上下文信息，被输入到大语言模型中，旨在激发模型依据预训练知识与内在推理机制，生成符合预期的文本回应。

像 LLaMA^[13-14]、Qwen^[15]和 GPT-4^[16]。这样的大语言模型(LLMs)，通过利用广泛的内部知识提升了推理能力。各种基于提示^[3]的方法进一步增强了这些能力。例如，思维

链(CoT)^[17]。Huang 等人^[60]通过将复杂任务分解为可处理的步骤来改进推理，在数学推理^[18]和逻辑推理^[19]等领域表现出色。像自动思维链(Auto-CoT)^[20]零样本思维链(Zero-Shot-CoT)^[21]和复杂思维链(Complex-CoT)^[22]等变体进一步优化了这种方法。诸如思维树(ToT)^[23]和思维图(GoT)^[24]这样的框架拓宽了大语言模型推理的范围。除此之外，还有将 LLMs 作为验证器^[55]的研究。

1.2.3 基于大语言模型的智能体

随着大语言模型的快速发展，基于大语言模型构建的智能体逐渐成为人工智能领域的重要研究方向。这类智能体以大语言模型为核心驱动，结合感知、决策和行动等模块，展现出强大的语言理解、推理和交互能力，为复杂任务的处理提供了新的解决方案。

从架构设计来看，基于大语言模型的智能体通常以预训练语言模型为基础，通过整合外部工具和功能模块，实现对多样化任务的智能化处理。在感知层面，智能体能够通过多种数据接口，如文本输入、传感器数据等，获取环境信息，并利用大语言模型的语言理解能力对信息进行语义解析和情境建模。在决策层面，智能体依托大语言模型的推理能力，结合任务目标与环境状态，生成合理的行动策略。在行动层面，智能体通过调用外部工具或执行具体操作，将决策转化为实际行为，形成完整的“感知-决策-行动”闭环。

与传统基于规则或强化学习的智能体相比，基于大语言模型的智能体具有显著优势。一方面，大语言模型强大的语义理解和推理能力使其能够灵活适应多样化任务场景，减少对人工规则设计的依赖；另一方面，智能体可以通过与环境的交互不断学习和优化，提升任务执行的准确性和适应性。目前，基于大语言模型的智能体已在对话系统、机器人控制、智能助手等领域展现出广阔的应用前景，为人工智能的发展注入了新的活力。例如，Park 等人^[62]开创了一个由大语言模型驱动的智能体框架，用于在交互场景中模拟人类行为，这充分凸显了大语言模型在模拟复杂社会交互和决策方面的潜力。BabyAGI 作为一个语言模型，它能够与任务列表进行交互，根据预先设定的目标自动完成任务的生成、排序和执行。Auto-GPT 利用 GPT-4 来实现人工智能的“思考”，并通过执行命令自主尝试达成特定目标，进一步拓展了人工智能能力的边界。在本文中，引入智能体的概念，目的是增强大语言模型的推理能力，从而提升其在知识图谱问答任务中的性能。

1.2.4 基于大语言模型的知识图谱问答

检索增强生成(RAG)技术^[70-73]在知识图谱(KGs)与大语言模型的融合中扮演着关键角色。知识图谱作为一种结构化的外部知识源，能为大语言模型提供丰富、准确的事实

知识，二者通过 RAG 技术相结合，在增强大模型推理能力和减少幻觉^[46-48]现象方面成效显著。

RAG 的核心机制是在大语言模型生成回答时，从外部知识源，如知识图谱，检索相关信息，并将其融入到生成过程中。在知识图谱中，知识以实体和关系构成的三元组形式存储，像“牛顿-提出-万有引力定律”这样的结构，清晰呈现了实体间的关联。当大语言模型接收到问题，比如“牛顿的主要科学成就有哪些”，首先对问题进行解析，提取关键实体“牛顿”和核心需求“主要科学成就”。在此基础上，借助 RAG 技术，依据这些关键信息在知识图谱中进行检索，得到牛顿在万有引力定律、微积分等方面的成就信息。这些知识反馈给大语言模型，作为生成答案的重要依据。如此一来，大语言模型不再仅仅依赖自身训练数据中的模糊知识，而是基于知识图谱提供的准确事实进行回答，从而有效避免了如遗漏重要成就、错误阐述成就内容等幻觉现象，生成的回答内容更准确、丰富，逻辑也更加连贯。

将知识图谱(KGs)与大语言模型^[40-41]相结合，可增强推理能力并减少幻觉现象。与诸如仅依赖大语言模型内部知识的思维链等方法不同，整合知识图谱能够获取结构化的外部知识^[25]。像 ToG^[26]、Interactive-KBQA^[27]和 StructGPT^[4]等方法，实现了大语言模型与知识图谱之间的逐步交互。诸如 RoG^[6]以及其他方法^[30]，利用预训练或微调的大语言模型为知识图谱问答生成检索信息。此外，像 UniKGQA^[32]和 KG-CoT^[31]等方法，需要训练特定模型来进行知识图谱信息检索，这进一步增加了其实施的复杂性。

1.3 研究内容和主要工作

本研究聚焦于知识图谱问答(KGQA)任务中的核心挑战，即如何准确回答复杂多跳问题。为了解决这一问题，本研究将大语言模型引入 KGQA 任务，重点解决推理链解析不准确和相关知识检索不全面的问题。当前用大语言模型做 KGQA 的时候普遍采用两种技术路线：一是基于推理路径规划的技术路线，即通过 LLMs 构建从问题到答案的多跳推理路径，利用知识图谱中的结构信息逐步推导出最终答案；二是基于信息检索的技术路线，即通过 LLMs 从知识图谱中检索与问题相关的实体和关系信息，并对其进行整合与推理以生成准确答案。针对上述两个技术路线，分别进行了探索。

首先，本研究基于推理路径规划思路，针对复杂多跳问题中推理链解析不准确的问题，提出了基于自我验证增强的图谱问答智能体(Self-Verification Enhanced Question Answering Agent, Self-VEQA Agent)。该方法旨在利用大语言模型生成高效的推理路径，并通过设计问答智能体(QA Agent)与验证智能体(Ver Agent)，构建了一个多轮迭代验证机制，以优化推理路径和答案的生成过程。验证智能体对问答智能体生成的推理路径及

结果进行判断并反馈修改意见,直至获得正确答案或达到最大迭代次数。通过多轮迭代,Self-VEQA 逐步修正错误,减少路径规划错误,并通过记忆机制积累历史经验,提升模型在类似问题上的表现。实验表明,该框架无需复杂微调,在 MetaQA 和 WebQSP 数据集上显著优于传统 KGQA 方法及多数同类方法,证明了其在复杂推理任务中的优越性。

其次,本研究基于信息检索的思路,针对复杂多跳问题中因问题复杂性导致检索不全面的问题,提出了基于迭代多跳知识检索与推理的图谱问答模型(Iterative Multi-hop Knowledge Retrieval and Reasoning, Iter-MKGRR)。该方法旨在利用大语言模型高效检索知识图谱并进行推理。通过问题分解策略将复杂多跳问题拆解为单跳子问题,降低了模型的检索和推理难度;结合迭代的知识图谱检索机制,逐步定位相关三元组,并引入知识整合技术,将子问题答案与当前问题融合,构建连贯的上下文信息,从而提升三元组检索的准确性。此外,该方法利用语义扩充技术深度捕捉问题意图,进一步优化检索效果。最后,充分发挥大语言模型的推理能力,结合检索的知识与内部知识生成准确答案。实验结果表明,该框架在多跳问题上的表现显著优于传统方法,尤其是在处理复杂问题时展现了更强的鲁棒性和准确性。

这两个框架虽然在技术路线上各有侧重,但共同目标是通过引入大语言模型,解决 KGQA 任务中的核心挑战。Self-VEQA Agent 通过多轮迭代验证机制和记忆机制,解决复杂多跳问题中推理链解析不准确的问题,进而提升复杂多跳问题的推理能力;而 Iter-MKGRR 则通过问题分解、知识整合和迭代检索,解决复杂多跳问题中检索不全面这一问题,进而提升复杂多跳问题的推理能力。实验结果表明,这两种框架在多个数据集上均取得了显著优于传统方法的性能,为 KGQA 任务的研究与应用提供了新的思路和方法。

1.4 论文主要结构

(1) 第一章为绪论部分,介绍知识问答任务的研究背景及现状,简要介绍主流方法的分类,并对比其优缺点,最后简要介绍本文主要工作及文章结构。

(2) 第二章介绍本文所使用的相关技术,着重介绍大语言模型和提示工程等技术。

(3) 第三章介绍基于大语言模型的智能体算法,详细叙述基于问答和验证智能体的设计,随后展示实验数据并分析实验结果。

(4) 第四章介绍基于迭代多跳知识检索与推理的图谱问答模型,对于复杂多跳问题,对其进行问题分解之后,创新地将每一步的答案与子问题做融合,再进行下一步检索。最后展示实验数据并分析实验结果。

(5) 最后为结论部分,总结本文研究工作,指出不足之处,并对未来工作进行展望。

2 相关理论与技术

本章首先介绍知识图谱涉及的基本定义。然后介绍本文涉及的相关理论基础，包括大语言模型、提示工程等。

2.1 基本定义

2.1.1 知识图谱

知识图谱表示结构信息 $G = (E, R)$ ， E 表示实体集， R 表示的是关系集。每个关系 $r \in R$ ，连接两个实体 (e_i, e_j) ，其中 $e_i, e_j \in E$ 。一个知识图谱 G 的示例如图 2.1 所示， E 包含诸如“盗梦空间”、“泰坦尼克号”、“英国”等实体，而关系 r 则描述了实体之间的关联。例如，连接“盗梦空间”与“2010 年”的关系 r 为“上映年份”。这种结构化的表示方式能够清晰地捕捉实体之间的语义关系，为知识推理和问答任务提供支持。

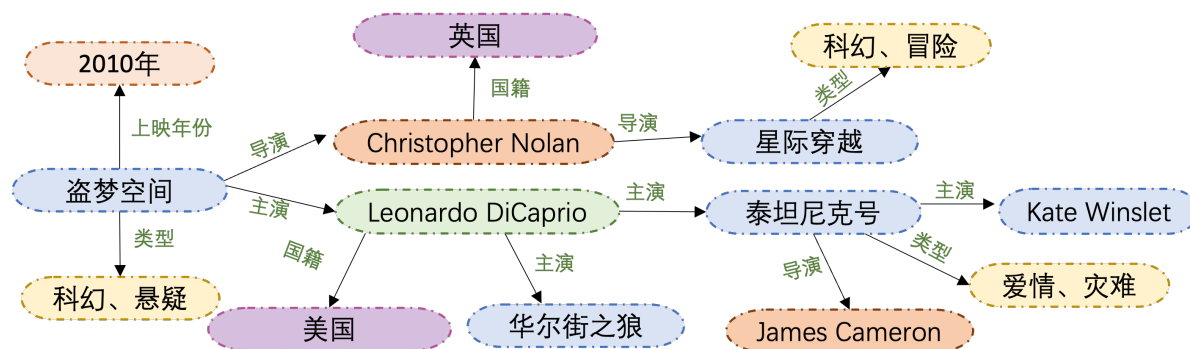


图 2.1 知识图谱

Fig. 2.1 Knowledge Graph

2.1.2 知识问答

知识问答的主要目标是使用知识图谱中的信息 G ，回答问题。通过输入问题 Q ，生成对应的答案。这一过程可以形式化表示为公式为 $f: A = f(Q, G)$ 。其中 A 是生成的答案。具体来说，系统通过分析问题 Q ，从知识图谱 G 中提取相关信息，并生成准确的答案 A 。以下通过一个示例详细说明这一过程。假设问题 Q 为：“盗梦空间的导演还导演过什么电影？”。对于这个问题，首先确定主题实体“盗梦空间”，然后通过图 2.2 中灰色阴影注释的推理链回答该问题。具体步骤如下：首先，系统从问题 Q 中提取主题实体“盗梦空间”；在此基础上，在知识图谱 G 中，找到与“盗梦空间”相关联的导演“Christopher Nolan”；然后，从“Christopher Nolan”出发，检索其导演的其他电影，也就是“星际穿越”，生成最终答案。通过这种方式，知识问答任务能够高效地利用知识图谱中的结

构化信息，完成复杂问题的推理与回答。

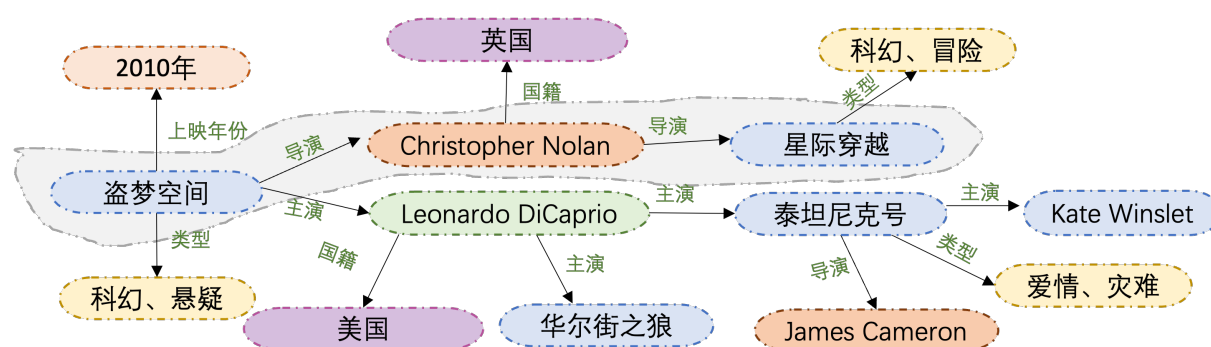


图 2.2 知识问答示例

Fig. 2.2 An Example of QA on KG

2.2 相关技术

2.2.1 大语言模型

在人工智能研究领域快速发展的背景下，大语言模型(Large Language Models, LLMs)已成为自然语言处理(Natural Language Processing, NLP)技术创新的核心驱动力，为 KGQA 等多个领域的研究与应用带来了新的思路。

大语言模型^[11-12]的底层架构构建于深度神经网络之上，其中，Transformer 架构凭借其架构设计脱颖而出，成为主流模型。更详细的结构可见图 2.3。从更深入的底层原理来看，Transformer 架构主要由编码器(Encoder)和解码器(Decoder)两大部分构成。编码器负责对输入文本进行特征编码，将自然语言文本转化为模型能够理解与处理的高维向量表示。它包含多个相同的层，每层又由多头注意力机制(Multi-Head Attention)与前馈神经网络(Feed-Forward Neural Network)组成。多头注意力机制通过线性变换将输入向量映射至多个子空间，各子空间内的注意力头并行捕捉文本不同层面的语义关联，有效避免了传统架构的局限性，使得模型能精准聚焦文本关键信息，就如同在剖析一篇涉及多学科交叉研究的论文时，基于 Transformer 的大语言模型能够精准解析开篇研究背景所引用的基础理论与后续实验论证、结论推导环节的内在逻辑连贯性，为知识理解与应用筑牢根基。而前馈神经网络则进一步对多头注意力机制输出的特征进行非线性变换，增强模型的表达能力。

解码器在生成文本时发挥关键作用，它同样由多层结构组成，除包含与编码器类似的多头注意力机制和前馈神经网络外，还引入了掩码多头注意力机制(Masked Multi-Head Attention)，该机制在训练过程中通过掩盖未来信息，确保模型生成文本时仅依赖已生成的前文信息，避免信息泄露，从而有序地逐字或逐词生成连贯合理的文本输出。

相较于传统循环神经网络(RNN)及其变体在处理长序列文本时所陷入的梯度消失或梯度爆炸等长期依赖困境,Transformer 架构实现了对输入文本序列并行化、多维度的特征提取,极大提升了模型训练效率与性能。

模型训练数据来源广泛多元,涵盖互联网全域文本,包括社交媒体动态、知识问答^[46]社区讨论等碎片化、口语化信息,以及出版书籍中的规范学术论述、经典文学创作,亦包括实时新闻资讯与严谨的高考知识库等。在数量级上,数十亿甚至上百亿的文本 tokens 经过系统采集和预处理后,被输入到模型训练流程中,以推动模型深入学习自然语言的语法规则、语义逻辑以及灵活适应不同语境的能力。以 GPT 系列模型为例,OpenAI 投入了大量资源组建跨领域的数据采集与标注团队,通过复杂的数据清洗、分类和结构化处理流程,确保训练数据涵盖自然科学、人文社科、工程技术等多个领域的知识,包括学术专著、科普文章和日常交流等多种体裁。这种全面的数据支持使模型具备了广泛的知识泛化能力。无论是应对日常闲聊场景,如提供旅游出行建议;还是处理专业学术领域的深度问题,如解释量子计算中的前沿算法或分析历史事件的复杂成因;甚至是进行文学艺术创作,如生成风格多样的诗歌或小说,模型都能基于学习到的内容,生成逻辑清晰、语义合理的文本输出。

参数规模是大语言模型的一个关键特征,随着参数数量从百亿级扩展到千亿级,甚至向万亿级迈进,模型的表征学习能力得到了显著增强。在模型训练过程中,海量参数通过随机梯度下降(SGD)及其优化算法,如 Adagrad、Adadelta 等,围绕损失函数最小化的目标不断迭代更新。训练过程依赖于大规模分布式计算集群,利用数以万计的图形处理单元(GPU)或专用人工智能芯片,支持复杂的矩阵运算和参数优化。经过长时间、高能耗的迭代训练,模型逐渐收敛到性能最优状态,能够适应自然语言任务的复杂性和多样性,有效应对文本分类、情感分析、机器翻译等不同任务的挑战。

在知识问答任务^[51]中,大语言模型表现出的优势,显著提升了任务性能。首先,由于模型在训练过程中吸收了海量数据,积累了广泛的知识储备,涵盖了多个学科领域和日常生活常识。例如,当被问及物理学中的“光的折射原理”时,模型能够迅速调用从光学教材、科普文章和相关学术研究中学习到的知识,提供准确且详细的解释,包括折射定律的数学表达式和实验验证案例等。其次,Transformer 架构的多头注意力机制使模型能够同时关注问题中的关键词、语义关系和上下文语境。例如,对于“在工业 4.0 时代,人工智能如何与制造业融合以提升生产效率”这类复杂且跨领域的问题,模型能够快速关联人工智能技术特性、制造业流程优化要点以及两者结合的实际案例等多方面信息,提供全面且深入的解答。此外,大规模的参数使模型具备强大的拟合能力,能够适应不同风格和难度的问题表述。无论是简单直接的日常知识提问,还是专业复杂的学术问题,

模型都能根据问题的特点动态调整内部参数，生成符合需求的答案。

预训练、微调策略以及提示工程是大语言模型实用化发展的核心路径。在预训练阶段，模型通过海量无标注文本进行学习，系统性地掌握语言的基本规则，包括语法、语义和语用，从而形成通用的语言理解和生成能力。当面向具体任务时，微调策略发挥作用。例如，在情感分类任务中，使用标注精细的小规模情感数据集对预训练模型的全连接层或特定中间层参数进行优化；在机器翻译任务中，则利用平行语料库帮助模型适应不同语言的结构差异。对于知识问答任务，可以通过特定领域的问答数据集进行微调，使模型更好地掌握该领域的专业术语、问题模式和回答方式，从而在目标领域的问答任务中表现更加出色。这种从通用语言学习到专用任务优化的过程，为自然语言处理任务的定制化和高效化提供了创新解决方案，促进了学术研究与产业应用的深度融合。此外，提示工程作为一种新兴的技术手段，进一步增强了模型在特定任务中的表现。通过设计巧妙的提示词或指令，可以引导模型生成更符合预期的输出，从而减少对大规模标注数据的依赖，降低模型优化的成本。

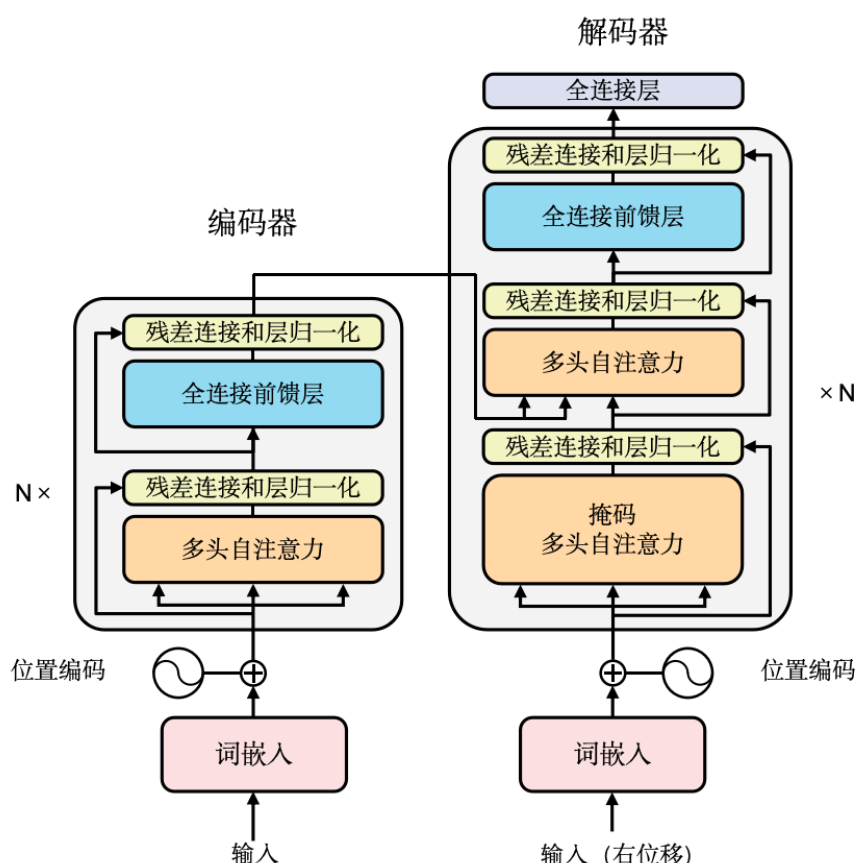


图 2.3 Transformer 架构图

Fig. 2.3 Structure of Transformer

2.2.2 检索增强生成

在当代人工智能技术迅猛发展、持续迭代的大背景下，大语言模型成为学术研究与产业实践领域共同关注的重点。然而，大语言模型在应用过程中，常面临幻觉问题和知识缺失等困境，严重影响其输出的可靠性与实用性。

在大语言模型的知识增强技术^[71]关键研究范畴内，检索增强生成(Retrieval-Augmented Generation, RAG)技术愈发重要。其凭借创新性架构与卓越应用效能，成为扩展模型知识范围、提升输出文本质量的关键手段，帮助大语言模型突破现有知识限制，向更高智能水平发展，探索新的可能性。详细的对比，可见图 2.4。

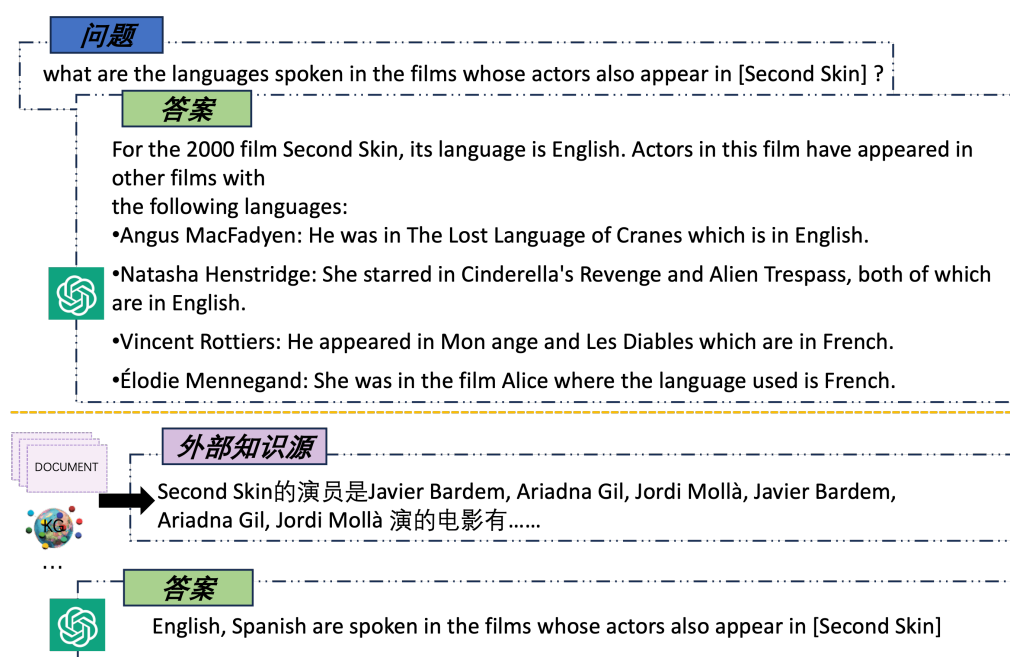


图 2.4 LLMs 直接回答问题和使用知识图谱问答

Fig. 2.4 LLMs Directly Answer Questions and Answer Questions on KG

RAG^[29]的核心架构结合了信息检索和模型推理两部分，协同工作。当输入文本后，系统会在外部知识源中检索相关信息。知识图谱作为一种结构化的知识库，以图的形式存储实体、关系和属性，节点表示实体，边表示实体间的关系，能够发挥重要作用。在检索过程中，知识图谱作为重要的外部知识源，帮助精准定位与输入文本相关的实体及详细信息。例如，在处理科技文献检索时，面对新兴技术术语，知识图谱能快速关联到该技术的研发团队、应用领域和相关专利等信息。这些信息与原始文本一起输入生成模块，生成模块利用大语言模型的推理能力，将结构化知识融入回复中，使输出文本逻辑连贯、信息准确，且具有更强的知识深度和关联性，为用户提供全面、专业的解答，缓解知识缺失问题。此外，知识图谱中的结构化信息具有较高的准确性和权威性，来源多

为严谨的学术研究和行业实践。这种结构化知识能够约束和校准模型，减少幻觉问题。例如，在历史知识问答中，模型若仅依赖内部知识，可能会混淆历史事件细节或人物关系，给出错误答案。但引入知识图谱后，模型能够依据清晰的历史人物关联和事件因果脉络，输出更可靠的答案。

相较于传统单纯依赖模型内部参数化知识的方式，RAG 技术^[35]有诸多显著优势，给大语言模型知识应用场景带来变革。其一，知识更新即时性强，当今知识爆炸式增长，各领域迭代迅速，尤其科技前沿、金融市场等关键领域，传统模型因架构局限内部知识更新滞后，难以满足用户对前沿资讯需求，而 RAG 技术^[61]只需适时更新外部知识源，模型就能迅速捕捉最新学术动态。其二，知识精准性有保障，在对知识精准度要求极高、容错率低的专业学术场景，微小差错可能引发严重后果，RAG 技术直接从严格筛选、来源可靠权威的外部知识源提取信息，这些知识源汇聚全球顶尖专家研究心血、临床实践经验与经典学术文献。如在医学咨询情境，用药指导、病症诊断等关键信息直接源自专业医学数据库，经专家核验、临床试验论证，为患者提供的建议精准无误，保障患者的健康。其三，知识的可解释性显著提升，在学术研究、法律咨询等严谨专业领域，知识来源依据至关重要，RAG 生成的回复能清晰展示所依据知识源头，用户若对答案存疑，可便捷回溯原始资料详细查证，这让用户在追求学术真理、严谨治学过程中更自信从容，增强对模型输出信任度，促使模型在高端专业领域充分发挥效能。

综上所述，检索增强生成(RAG)作为知识增强技术领域的重要方向，在与知识图谱结合后，为大语言模型的知识拓展提供了一条极具潜力的路径。尽管 RAG 在发展中仍面临诸多挑战，例如知识源的整合优化、算法效率的提升以及多场景适配性的增强等，但随着技术的持续迭代与优化，这些难题有望逐步得到解决。这将推动大语言模型在知识驱动型智能应用领域稳步前进，取得更加显著的成果。

2.2.3 提示工程

提示工程(Prompt Engineering)^[9]是指在与大语言模型交互过程中，精心设计输入文本(即提示)以引导模型生成符合预期输出的技术，它处于人与大语言模型沟通的关键环节。从底层原理来看，大语言模型是基于海量文本数据的学习构建起复杂的神经网络架构，模型通过对这些数据中的语法、语义、逻辑等模式进行捕捉与归纳，形成自身的知识体系。当输入精心设计的提示时，相当于为模型激活特定知识区域、引导其推理路径提供了触发条件，一个精妙设计的提示能够挖掘模型的潜在能力，使其精准地完成诸如文本创作、知识问答、代码生成等多样化任务。在当前大语言模型广泛应用的场景下，提示工程的质量直接关系到模型使用的效果与效率，是解锁大语言模型强大功能的关键步骤。

提示主要包含几个基本组成要素,指令能明确告知模型需要执行的任务,例如“请写一篇关于人工智能发展趋势的综述文章”,从原理上讲,指令就是给模型的一个明确目标导向,让其依据已学习到的知识体系,快速定位到与人工智能相关的内容模块,并按照综述文章的组织逻辑进行输出,让模型知晓行动方向;上下文可为模型提供相关背景信息,像是要模型续写科幻小说,提前给出前文情节、人物设定等上下文,这是因为模型在学习过程中对文本的连贯性有一定认知模式,补充的上下文相当于为其续接故事提供衔接线索,帮助模型延续故事逻辑;当期望模型生成特定格式内容时,示例的作用就很显著,如要求模型生成古诗,给出一首古诗范例并注明格律要求,模型便仿写,这是利用了模型对相似模式的模仿学习能力,范例为其提供了直观的格式、韵律等参考模板。

提示工程也有技巧,通过角色设定,如设定“你是一位资深历史学家”等角色,底层原理在于触发模型调用与该角色专业领域匹配的知识子集,让模型代入专业视角,在回答历史问题时,模型会采用更严谨、专业的口吻,提供更具深度的见解。采用多轮对话式提示,模拟人与人的对话流程,首轮提示获取初步结果,后续基于此不断追问、细化,像首轮让模型规划旅行大纲,第二轮针对景点细节、交通安排等进一步提问,逐步完善方案,这契合了模型逐步推理、优化输出的特性,每一轮新的提示都是在已有输出基础上的补充信息输入,引导模型持续优化结果。还可以在指令中添加“简洁地”“详细且深入地”等限定词与修饰语,精准调控模型输出的详略程度与风格特色,这是因为模型能够解析这些指令中的修饰信息,并据此调整知识提取与文本生成的策略。

不过,提示工程同样面临挑战。一方面,模型可能存在理解偏差,尽管精心设计,模型仍可能误解提示含义,尤其面对复杂、抽象任务,如隐喻、双关语情境下,这是由于模型虽然学习了海量文本,但对一些隐含的、需要深度语义理解和文化背景依托的表达,还难以精准把握,容易给出偏离预期的回答;另一方面,存在敏感性与偏见问题,若提示涉及敏感社会议题或隐含偏见信息,模型输出可能引发争议,这是因为模型学习的数据本身可能存在一些社会固有偏见的印记,需要严格把控提示内容,避免不良导向;此外,还有通用性局限,针对特定领域优化的提示,在跨领域应用时效果大打折扣,这是由于模型在特定领域学习的知识结构与通用领域有差异,提示工程需兼顾通用性与专业性的平衡。

3 基于自我验证增强的图谱问答智能体

3.1 引言

本研究聚焦于基于推理路径优化的技术,旨在解决回答复杂多跳^[34]问题推理链解析不准确的问题,进而提升 KGQA 任务的性能。在不进行微调的前提下,现有主流方法主要利用大语言模型生成 SPARQL^[12]语句来实现知识图谱问答。SPARQL^[38]作为知识图谱查询的有效工具,对推理链解析查询。原本期望借助大语言模型强大的生成能力实现高效精准查询。但实际情况是,大语言模型在理解复杂语义与语法结构方面存在固有局限。例如,生成的 SPARQL^[67]语句频繁出现格式错误,无法满足查询规范,又或是在解析关系链时出现理解偏差,最终导致知识图谱查询失败,无法获取正确答案。

除此之外,已有部分研究工作尝试利用大语言模型进行验证,期望借此强化其推理能力,不过当前该领域仍有较大的提升空间。比如,Dammu 等人^[50]引入了验证概念,基于少样本方法生成伪图,试图改善问答效果。但遗憾的是,其整个验证流程仅执行了一次,这种单次验证难以全面深入地探究问题,所得到的验证结果大概率并非最优解,仍存在诸多可改进之处。

为有效应对上述挑战,本研究创新性地引入了基于自我验证增强的图谱问答智能体(Self-VEQA Agent)。本研究这一方法核心在于运用自主自我验证理念,针对性地解决大语言模型推理能力不足的问题。与以往类似研究相比^[39],本研究的方法采用多个智能体协同运作,开启多轮自主验证模式。具体而言,在基础大语言模型之上,构建了两个智能体:问答智能体(QA Agent)和验证智能体(Ver Agent)。

问答智能体作为基础的问答模块,主要用于理解问题并初步给出答案。为了解决生成的关系链不准确的问题,本文提出的验证智能体对问答智能体所给出的答案进行验证,重点排查可能存在的推理错误,确保答案的准确性与可靠性。这两个智能体紧密协作,通过多轮反复检查与调整答案来进行迭代优化。直至获取最终满意结果或达到预先设定的最大迭代次数。不仅如此,该方法还融入了记忆机制,使其具备动态进化能力。随着模型持续执行任务、积累经验,问答智能体在后续任务中的准确性将逐步提高。

3.2 方法

3.2.1 模型概述

Self-VEQA Agent 包含四个模块:跳数预测模块、问答模块、验证模块和记忆模块。具体框架如图 3.1 所示。在跳数预测模块中,Self-VEQA Agent 首先获取当前问题的跳

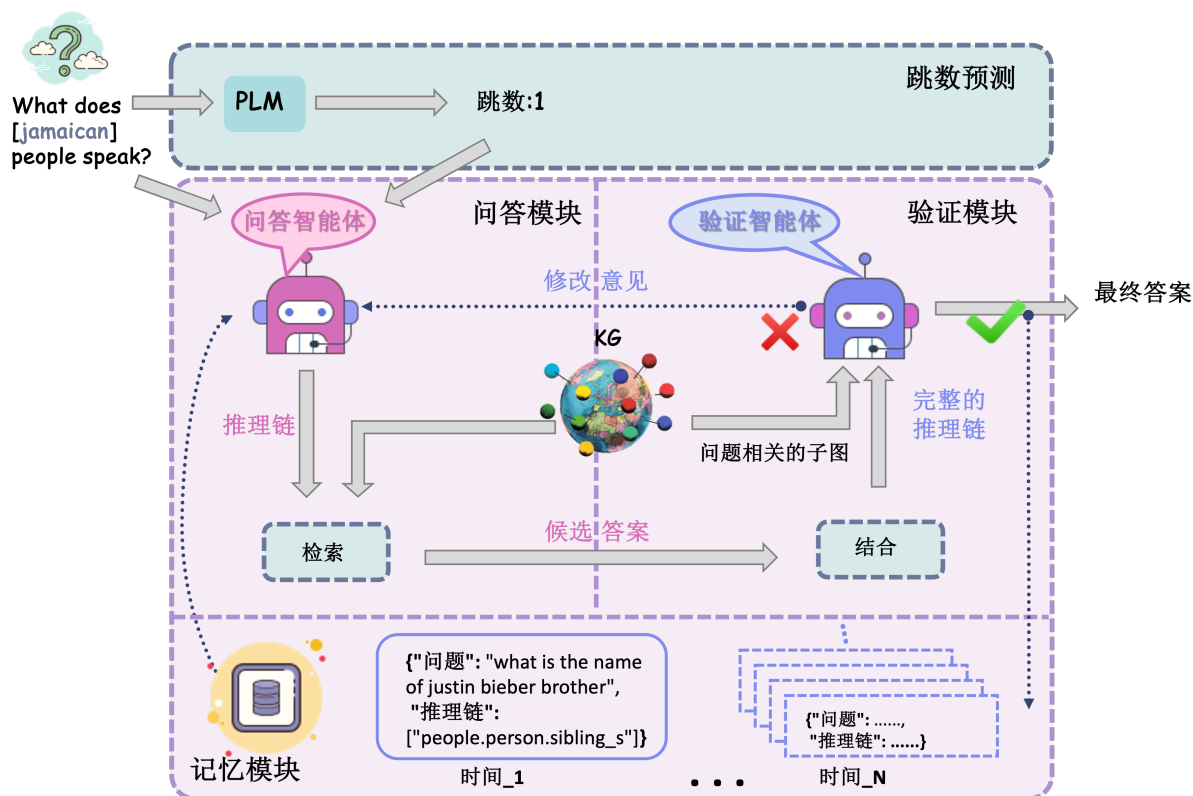


图 3.1 Self-VEQA Agent 框架图

Fig. 3.1 Structure of Self-VEQA Agent

数。在问答模块中，QA Agent 无需任何训练或微调，基于当前问题生成 n 条推理链，然后检索知识图谱以生成 n 个候选答案。利用这些候选答案和头实体，可以得到 n 条完整的推理链。

接下来，在验证模块中，当前问题的相关子图谱(subKG)被用作 Ver Agent 的外部知识与推理来源。由于标记数量有限，验证智能体首先需要筛选出与输入问题最相关的子图谱信息。这一筛选过程通过计算问题与子图谱中各个节点的语义相似度来实现。本研究使用 all-MiniLM-L6-v2 模型对问题和子图谱节点进行向量化表示，并基于余弦相似度筛选出最相关的子图谱信息。通过这种方式，验证智能体能够聚焦于与问题高度相关的知识，从而提高推理的效率和准确性。验证智能体会根据子图谱信息验证这 n 条完整推理链的合理性。在验证过程中，验证智能体会与问答智能体进行交互。如果推理链中存在合理答案，那么输出即为正确答案。否则，会提供修改建议，说明当前推理链及答案为何无法解决当前问题以及如何修改。修改建议作为反馈提供给问答智能体，使其能够基于反馈生成更准确的候选推理链。

在记忆模块中，随着正确结果在记忆中不断累积，问答智能体通过将过往成功案例作为少样本进行学习。这一过程类似于人类学习，即个体通过实践学习、积累经验，并持续提升自身能力。其中图 3.2 为具体的算法流程。

算法3.1: Self-VEQA 算法

```

输入:  $Q$ 
输出: 最终答案
1:  $h_Q \leftarrow$  跳数预测模块( $Q$ )
2:  $C, A \leftarrow$  问答模块( $Q, h_Q$ , 记忆模块)
3:  $result \leftarrow$  验证模块( $C, A, subKG$ )
4: if  $result$  是答案, then
5:    $C^* \leftarrow$  确定正确推理链
6:   记忆模块( $C^*, Q$ )
7:   return  $result$ 
8: else
9:    $modification_{suggestion} \leftarrow$  验证模块( $C, A, subKG$ )
10:   $C, A \leftarrow$  问答模块( $Q, h_Q$ , 记忆模块,  $modification_{suggestion}$ )
11:   $result \leftarrow$  验证模块( $C, A, subKG$ )
12:  if  $result$  是答案 then
13:     $C^* \leftarrow$  确定正确推理链
14:    记忆模块( $C^*, Q$ )
15:    return  $result$ 
16:  else
17:    return  $result$  to 问答模块 (修改建议等信息)
18:  end if
19: end if

```

图 3.2 Self-VEQA Agent 算法流程

Fig. 3.2 Self-VEQA Agent Algorithm process

3.2.2 跳数预测模块

跳数预测，从本质上来说，是指针对多跳问题进行回答前，对所需跳数展开的预测。其核心目的在于通过这一预测，得到当前问题的推理路径的长度，进而提升整个问答系统的效率与准确性。这个过程本质上为一个分类任务，利用预训练语言模型(PLM)和一个简单的线性分类器来完成。通过提供跳数，它将帮助问答智能体更准确地确定推理链。在 WebQSP 数据集的训练集上对 bert-base-uncased^[42]和一个线性分类器进行微调，以进行跳数预测。

在公式 3.1 和公式 3.2 中， Q 代表问题； V_Q 则是对问题 Q 进行嵌入处理后的结果，这

一嵌入过程旨在将问题文本转化为模型能够高效处理的方式。在公式 3.1 和公式 3.2 中， Q 代表问题； V_Q 则是对问题 Q 进行嵌入处理后的结果，这一嵌入过程旨在将问题文本转化为模型能够高效处理的向量形式，以适配后续的计算流程；而 h_Q 便是经过模型运算最终预测得出的跳数。

$$V_Q = PLM(Q) \quad (3.1)$$

$$h_Q = \operatorname{argmax} P(h|V_Q), h \in 1, 2, \dots, H \quad (3.2)$$

3.2.3 问答模块

本研究的方法设计了一个问答智能体，它负责基于知识图谱 G 生成 n 条候选推理链。为了让问答智能体生成更准确的推理链 $C = \{c_1, c_2, \dots, c_n\}$ ，本研究采用了提示工程。提示 P 中使用的少样本提示是从训练集初始化的，后续是基于记忆模块 M 进行少样本选择。基于候选推理链，可以从知识图谱中获取初始答案 A 。具体可见图 3.3。

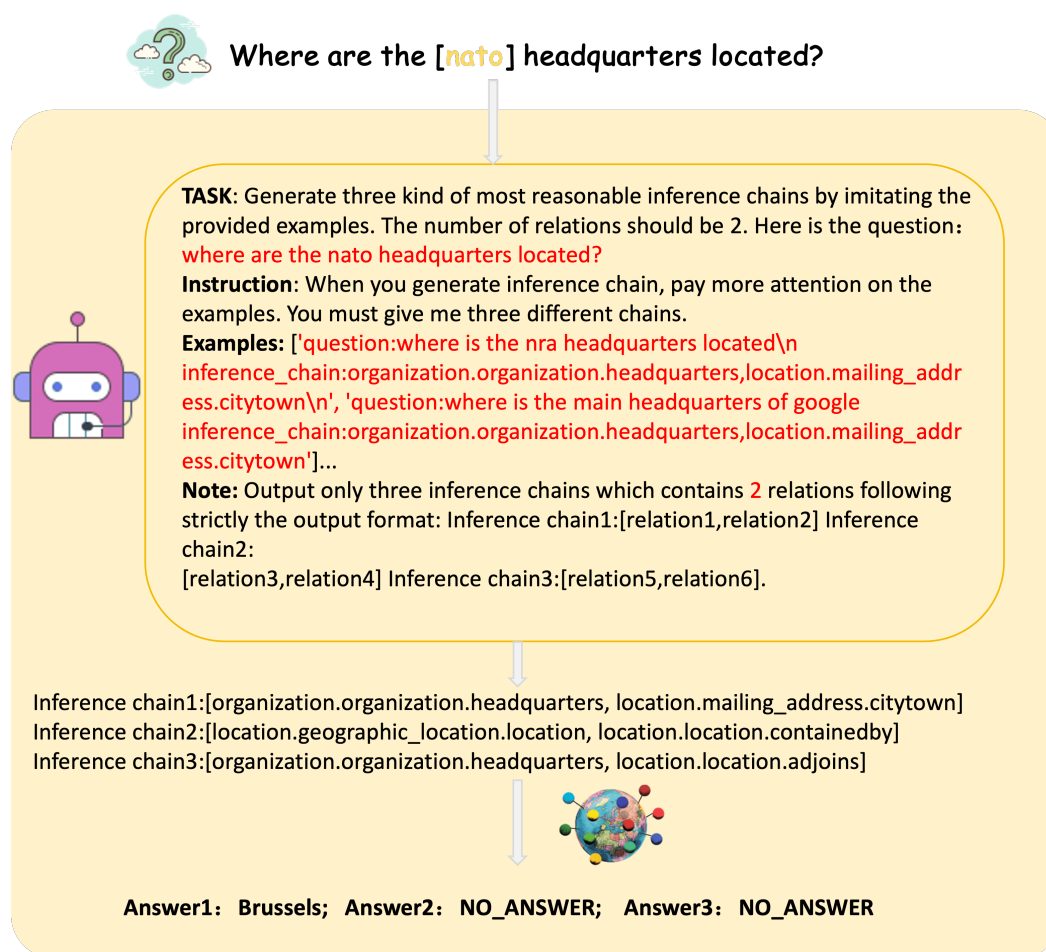


图 3.3 问答智能体 prompt

Fig. 3.3 Prompt of QA Agent

具体可见如下公式：

$$C = QA_Agent(G, P) \quad (3.3)$$

$$P = Initialize_Prompt(T) + Select_Few_Shot_Examples(M) \quad (3.4)$$

$$A = Retrieve_Answer(G, C) \quad (3.5)$$

问答智能体主要有两个能力：第一个是生成 n 条候选推理链，第二个是根据修改建议修改推理链。生成的 n 条候选推理链以及检索到的答案会由验证智能体进行验证。如果能从 n 条链中选出一条合理的，过程就结束。正确的推理链会被存储在记忆模块中。否则，问答智能体将根据验证智能体给出的修改建议 S 修改推理链，得到新的推理链 C' 。记忆模块为问答智能体提供经验，通过不断使用，持续丰富并提高其准确性。这展示了问答智能体的动态进化能力，因为随着每个完成的任务和积累的经验，其准确性会稳步提升。公式如下所示：

$$C' = QA_Agent_{modify}(C, S) \quad (3.6)$$

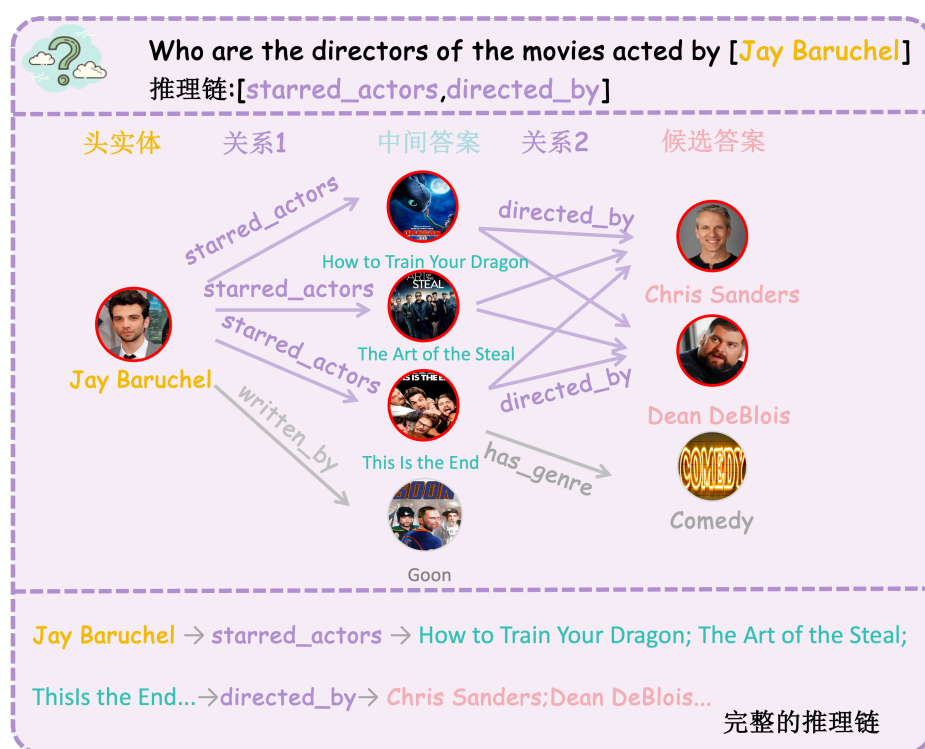


图 3.4 完整的推理链

Fig. 3.4 Integrated Inference Chain

为了更深入了解问答智能体的细节，从拆分问答智能体提示的各个部分开始，它主要由三个部分组成：任务、指令和注意事项，如下所示：

任务部分：主要目的是向问答智能体清晰传达其主要任务是什么。问题及其相应的跳数作为外部输入提供给问答智能体。

指令部分：这里的少样本提示来自记忆模块，使用了训练集用于初始化。少样本提示由问题和推理链组成。此外，这种设置展示了模型的动态进化能力，因为它不断从积累的记忆中学习。

注意事项部分：对输出格式以及具体输出标准提出严格要求。

基于此，问答智能体生成 n 条推理链。结合头实体(head entity)，可以通过在知识图谱中搜索确定 n 个候选答案。这里的头实体可以从数据集中获取。候选推理链和检索到的答案将被组合在一起，形成完整的推理链。基于此，可以得到 n 条完整的推理链和答案集。图 3.4 展示了一个具体的示例。这 n 条完整的推理链，连同答案集以及与问题相关的子图谱将输入到验证智能体中进行答案选择。这里，“相关子知识图谱”指的是知识图谱中专门与当前问题相关的一个子集。这个子集是通过与问题进行相似度计算确定的，这些计算有助于识别知识图谱中与当前查询密切相关的节点和关系。这里的相似度计算主要使用的是 sentence-transformers/all - MiniLM - L6 - v2。通过使用这个相关子知识图谱，本研究的方法可以有效地缩小处理验证所需的三元组范围，从而减少输入文本的总数并提高了效率。

3.2.4 验证模块

为了提高问答智能体生成答案的准确性，本研究设计了一个验证智能体(Ver Agent)。验证智能体有两个能力：第一是验证 n 条整合推理链 C 中是否有一条 c^* 能够回答问题；第二是如果验证智能体认为这些推理链都无法回答问题，则提供修改建议 S 。这个智能体主要利用大语言模型(LLMs)的推理能力，评估前一步生成的完整推理链的正确性。如果其中一条推理链是正确的，验证智能体就会选择并输出最终答案。否则，它会给出修改建议，解释为什么不能回答问题，并将修改建议传递给问答智能体以便重新生成答案。这个过程会一直重复，直到验证智能体选择出正确答案，或者达到最大轮数。

具体公式，如下所示：

$$c^* = Ver_Agent(C, A) \quad (3.7)$$

$$S = Ver_Agent_{suggest}(C, A) \quad (3.8)$$

为了更深入了解图 3.5 中的细节，从拆分验证智能体提示的各个部分开始，它主要由三个部分组成：任务、指令、注意事项。

任务部分：主要目的是向验证智能体清晰传达其主要任务是什么。这里 n 条整合推理链将被传递给验证智能体。

指令部分：详细说明了任务的具体方面，为验证智能体提供推理依据，是该智能体功能的核心。

注意事项部分：对输出格式以及具体输出标准提出严格要求。

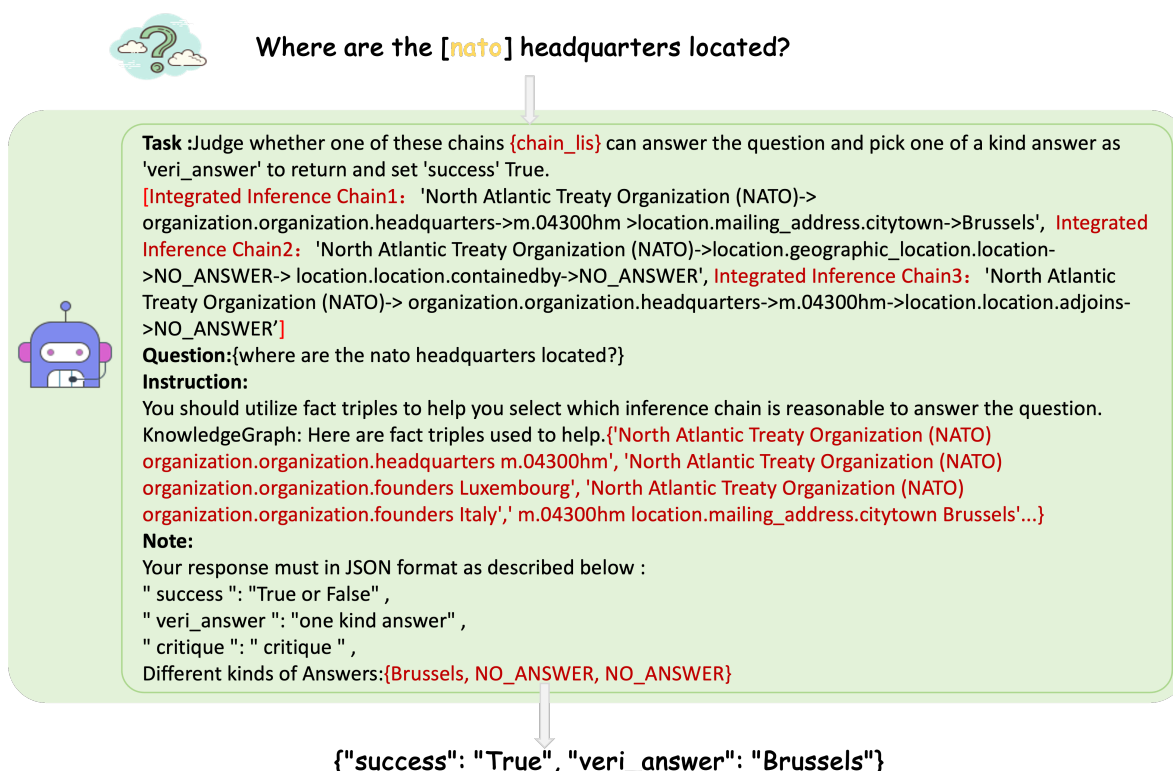


图 3.5 验证模块 prompt

Fig. 3.5 Prompt of Ver Agent

3.2.5 记忆模块

在当前的研究背景下，大语言模型展现出了强大的语言处理能力，然而其记忆机制存在一定的局限性。鉴于此，本文创新性地提出了记忆模块 M ，旨在作为大语言模型经验的有效存储空间，以弥补现有记忆机制的不足，并进一步优化知识图谱问答流程。

具体而言，记忆模块主要的功能是用于存储那些被验证智能体确认为正确的推理链以及对应的问题。在 Self-VEQA Agent 中，问答智能体首先会基于输入问题尝试生成推理链及答案候选项，随后这些初步结果会被递交至验证智能体进行审核。只有当验证智能体依据在 prompt 预先设计的标准、指令以及知识图谱中检索的信息，判断某一推理链 c^* 及相关问题解答准确无误时，它们才会被筛选并录入记忆模块 M 之中。具体公式，如下所示：

$$M \leftarrow M \cup \{c^*\} \quad (3.9)$$

这种基于正确性筛选的存储机制意义重大,随着时间的推移以及问答任务的持续推进,记忆模块逐步积累起丰富多样且可靠的经验,进而转化为问答智能体后续执行少样本示例选择任务时的经验库。当面对新的问题时,问答智能体不再需要从零开始进行推理与探索,而是能够快速地在记忆模块中检索与之相似的过往经验,借鉴已有的成功推理链与答案模式,从而极大地提高问题解决的效率与准确性。图 3.6,展示了更具体的一个交互过程。

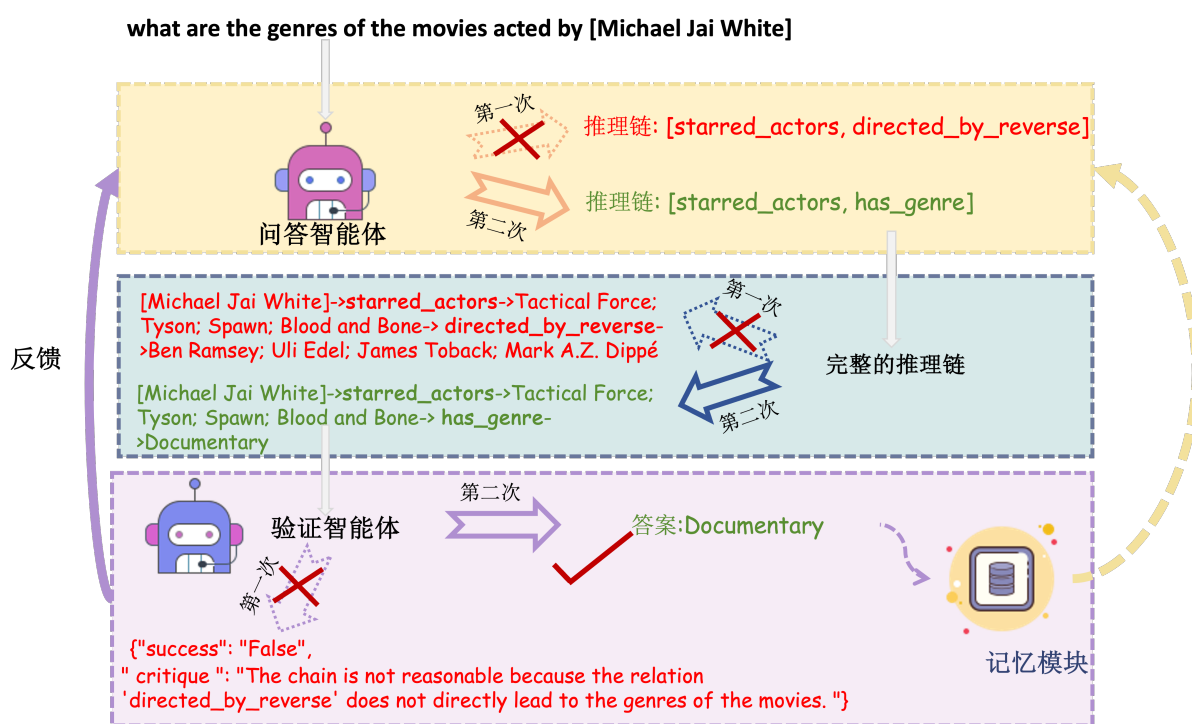


图 3.6 Self-VEQA Agent 案例分析

Fig. 3.6 Case Study of Self-VEQA Agent

3.3 实验

3.3.1 数据集

本研究在具有代表性的小规模和大规模知识图谱及其对应的知识图谱问答(KGQA)数据集上进行了评估。

(1) MetaQA^[63]数据集

MetaQA 是电影领域的一个大型 KGQA 数据集,其知识图谱包含 43000 个实体、9 种关系和 135000 个三元组。它包含 407000 个需要从头部实体进行 1 跳至 3 跳推理的问题。每个问题由一个头部实体、一个关系推理路径和答案实体组成。为了评估 Self-VEQA Agent 的多跳推理能力,在实验中重点关注 MetaQA 中的 2 跳和 3 跳数据集。

(2) WebQSP 数据集

WebQSP 是一个问题数量较少，但知识图谱规模较大的基准数据集。它包含 Freebase^[66]上最多 2 跳的问题，每个问题都有一个主题实体、约束条件、推理链和用于查找答案的 SPARQL 查询。使用了谷歌提供的最新 Freebase 数据转储，截至 2023 年，其中包含 31.2 亿个三元组。WebQSP 有 4737 个问题，但排除了 11 个没有标准答案的问题。

3.3.2 评估指标

在知识图谱问答(KGQA)领域的研究进程中，为了确保实验结果的科学性、可比性以及有效性，精准且适宜的评估指标选取至关重要。与众多先前在此领域深耕的研究工作保持一致，经过审慎考量与反复验证，最终决定采用 Hits@1 和 F1 这两个具有广泛认可度与代表性的指标作为衡量 Self-VEQA Agent 性能表现的关键标尺。

首先对于 Hits@1 指标，它在评估体系中扮演着直观且关键的角色。Hits@1 着重衡量的是在一系列问答测试样本中，模型所预测出排名第一的答案能够精准匹配正确答案的问题所占的实际百分比。通过对大量不同类型、不同领域问题的 Hits@1 值统计，能够快速且有效地了解智能体在初次尝试解答问题时的准确率水平，进而直观把握其核心性能表现。

$$Hits@1 = \frac{N_{correct}}{N_{total}} \times 100\% \quad (3.10)$$

其中， $N_{correct}$ 是预测排名第一且正确的答案数量； N_{total} 是总问题数量。

然而，知识图谱问答任务的复杂性决定了单一指标难以全面反映智能体的能力。考虑到现实世界中的问题往往存在多种合理的正确答案，尤其是在涉及知识图谱中的复杂关系查询以及多路径推理问题时，情况更是如此。基于此，本文还采用了 F1 指标，它在评估体系中发挥着不可或缺的补充作用。F1 指标充分考虑了所有可能的正确答案的覆盖范围，在预测响应的精确率和召回率之间寻找到平衡。精确率衡量的是智能体预测为正确的答案中，真正正确的答案所占的比例，它反映了模型预测的准确性；召回率则侧重于关注所有真实存在的正确答案中，被智能体成功预测出来的比例，体现了模型对正确答案的捕捉能力。F1 指标通过综合这两个方面，以一种更为全面、细致的方式评估智能体的性能。例如，在面对一个询问某部文学作品的作者、创作背景以及相关流派等多方面信息的复杂问题时，Self-VEQA Agent 可能会给出一组包含部分正确信息的答案，此时精确率会考量其中准确信息的占比，召回率则反映智能体是否涵盖了该问题所有关键的正确答案，而 F1 指标则将二者有机结合，给出一个综合考量后的评估值，使得对智能体在处理此类复杂多答案问题时的表现有更为精准的判断。具体公式，如下所示：

$$F1 = 2 * \frac{P * R}{P + R} \quad (3.11)$$

$$P = \frac{N_{correct}}{N_{predicted}} \quad (3.12)$$

$$R = \frac{N_{correct}}{N_{true}} \quad (3.13)$$

其中 $N_{correct}$ 是预测正确的答案数量。 $N_{predicted}$ 是模型预测的答案总数。 N_{true} 是真实存在的正确答案总数。

综上所述, Hits@1 和 F1 这两个评估指标相辅相成, 从不同维度、不同侧重点对 Self-VEQA Agent 的性能进行了全面而深入的剖析。它们不仅为本研究在实验过程中的模型对比、优化迭代提供了坚实的数据支撑, 同时也使得本研究能够与先前的相关研究在同一评估框架下进行有效对比。

3.3.3 基线模型

为了评估 Self-VEQA Agent 的有效性, 在 WebQSP 和 MetaQA 数据集上, 将其与知识图谱问答领域中一系列已有的基线模型进行了对比分析。这些基线模型可以分为不包含大语言模型的传统方法和基于大语言模型的方法。

(1) 传统方法基线: 包括 KV - Mem^[1]、EmbedKGQA^[2]、NSM^[33]、UniKGQA^[32]。所有这些基线模型都在 MetaQA 和 WebQSP 数据集上进行了评估。

(2) 基于大语言模型的基线: 本研究添加了 StructGPT^[4]、KnowledgeNavigator^[36]、ReasoningLM^[52]和 Readi - GPT3.5^[7]作为利用大语言模型的知识图谱问答任务的基线模型。

3.3.4 实现细节

本研究通过 OpenAI API 使用闭源的 GPT-3.5-turbo 模型。为了保证结果可复现, 温度参数设置为 0.0, 以消除生成过程中的随机性, 确保每次生成的输出具有确定性。上下文长度为 4096, 以充分利用模型的长文本处理能力, 同时避免因上下文过长导致的性能下降。每个输出序列最多 2000 个标记。在所有实验中, 问答智能体和验证智能体之间的最大交互次数设置为 2。

3.3.5 实验结果

表 3.1 展示了 Self-VEQA Agent 和基线模型在知识图谱问答数据集 MetaQA 上的性能。每个模块中的最佳结果以粗体显示。Δ表示在 MetaQA 上从两跳任务到三跳任务的 Hit@1 下降情况。

对于 MetaQA 2 跳数据集，该数据集相对简单，因此传统方法和基于大语言模型的方法都能取得相对较好的性能，很多模型的得分都达到了 99 以上。Self-VEQA Agent 的表现优于大多数传统方法和基于大语言模型的方法。不过值得注意的是，Self-VEQA Agent 在 MetaQA 2 跳问题上的表现不如 NSM。这主要是因为 MetaQA 数据集包含不到 300 个模板问题，但有超过 100000 个训练实例，这使得模型可以进行大量训练，从而取得较高的性能。而本研究方法主要依赖大语言模型，且没有针对特定的下游任务进行微调。

表 3.1 Hit@1 结果(MetaQA)
Tab.3.1 Hit@1 Results (MetaQA)

方法名	MetaQA 2-HOP	MetaQA 3-HOP	Δ
KV-Mem ^[1]	82.7	48.9	-33.8
EmbedKGQA ^[2]	98.8	94.8	-4.0
NSM ^[33]	99.9	98.9	-1.0
UniKGQA ^[32]	99.0	99.1	+0.1
Davinci-003 ^[56]	25.3	42.5	-12.2
ChatGPT	31.0	43.2	-12.2
GPT4	52.5	49.2	-3.3
StructGPT(ChatGPT) ^[4]	97.3	87.0	-10.3
KnowledgeNavigator ^[36]	99.5	95	-4.5
Readi-GPT3.5 ^[7]	99.9	99.2	-0.7
ReasoningLM ^[52]	98.3	92.7	-0.6
Self-VEQA Agent(Ours)	99.7	99.6	-0.1

MetaQA 2 跳与 3 跳问题对比，从实验结果可以看出，对于大多数模型，包括传统模型和基于大语言模型的模型，在 MetaQA 数据集上从 2 跳问题过渡到 3 跳问题时，性能往往会下降。这主要是因为在 MetaQA 数据集上，正确推理 3 跳问题比 2 跳问题更具挑战性。3 跳问题的推理链由三种关系组成，这种复杂性使得推导准确的顺序和关系更加困难。对于未进行微调的基于大语言模型的方法，如 KnowledgeNavigator 方法，随着跳数的增加，性能往往会下降。这是因为 KnowledgeNavigator 主要依赖知识增强方法，随着跳数的增加，所需的知识也呈指数级增长，从而增加了基于大语言模型推理的难度，导致与 2 跳问题相比性能下降。与其他将检索和推理视为不同阶段的方法不同，UniKGQA 方法为这两个过程提出了一个统一的模型，有效地将相关信息从检索阶段转

移到推理阶段，因此即使是 3 跳问题，其性能也能保持稳定。相比之下，本研究的方法在 MetaQA 数据集上从 2 跳问题过渡到 3 跳问题时，性能几乎没有变化。本研究的方法在 3 跳问题上的性能与 2 跳问题相比没有显著下降，主要是因为验证智能体设计，它可以验证生成的推理链和答案的合理性。此外，随着任务经验的积累，生成的推理链的错误率会随着时间下降，从而提高整体性能。

表 3.2 展示了 Self-VEQA Agent 和基线模型在知识图谱问答数据集 WebQSP 上的性能。每个模块中的最佳结果以粗体显示。

表 3.2 Hit@1 和 F1 结果(WebQSP)

Tab.3.2 Hit@1 和 F1(WebQSP)

方法名	F1	Hit@1
KV-Mem ^[1]	34.5	46.7
EmbedKGQA ^[2]	-	66.6
NSM ^[33]	62.8	68.7
UniKGQA ^[32]	70.2	75.1
Davinci-003 ^[56]	-	48.7
ChatGPT	-	61.2
GPT4	-	70.7
StructGPT(ChatGPT) ^[4]	-	72.6
KnowledgeNavigator ^[36]	-	83.5
Readi-GPT3.5 ^[7]	-	78.7
ReasoningLM ^[52]	71.0	78.5
AgentBench ^[59]	-	47.8
Self-VEQA Agent(Ours)	70.5	82.0

对于 WebQSP 数据集，WebQSP 数据集的关系链更为复杂，并且包含带有约束条件的问题，如前文所述。不过，该数据集问题的最大跳数为 2。因此，该数据集的主要挑战在于识别正确的关系链和相应的约束条件。本研究的方法明显优于传统模型，主要是因为与 MetaQA 相比，WebQSP 数据集包含更多的模板问题，但训练样本较少，而且一些关系链的名称相似且多样，这使得传统模型难以充分学习。与基于大语言模型的方法相比，本研究的方法也比大多数模型表现更好。这主要是因为不仅简单地利用大语言模型的内部知识或输入相关的外部知识，还提出了两个智能体相互交互和验证，从而提高了性能。该研究的方法在 WebQSP 数据集上比 KnowledgeNavigator 落后约 1.5%，主要

是因为 KnowledgeNavigator 使用知识增强技术提供更准确的事实三元组，从而为有约束条件的问题提供更好的答案。本研究的方法在 F1 指标上也略落后 ReasoningLM，ReasoningLM 专注于在知识图谱上进行推理，而使用少样本提示引导大语言模型生成关系链，并通过多轮验证来取得结果。

3.3.6 消融实验

为了证明 Self-VEQA Agent 内部各个模块的实际作用，本研究设计了消融实验。

表 3.3 消融实验(MetaQA)

Tab.3.3 Ablation experiments (MetaQA)

模型	MQA 2-HOP	MQA 3-HOP
S-VEQA	99.7	99.6
-mem	99.0	99.4
-ver	98.9	98.7

具体而言，本研究着重聚焦于验证智能体(Ver Agent)以及记忆模块这两个核心组件，通过分别将它们从 Self-VEQA Agent 架构中移除，以此来评估它们各自对整体智能体性能所产生的影响。详细的实验结果如表 3.3 和表 3.4 所示。其中 S-VEQA 表示的是本论文中提出的模型。-mem 表示去除记忆模块。-ver 表示去除验证模块。本研究所提出的 Self-VEQA 模型作为基准对照，其在不同数据集下展现出了卓越的性能表现。在表 3.3 中，对于 MQA 2 - HOP 数据集，Self-VEQA 模型 Hit@1 达到了 99.7；在 MQA 3 - HOP 数据集中，Hit@1 值达到 99.6。对模型模块进行消融，移除记忆模块后，模型性能出现了一定程度的下滑。在 MQA 2 - HOP 任务中，Hit@1 值降至 99.0；MQA 3 - HOP 任务里，该指标变为 99.4。这明确显示了记忆模块在模型运行中的重要作用，它帮助智能体存储、检索和连接关键信息。缺少这一模块会干扰模型在处理复杂问题时的信息流畅性和准确性。当移除验证智能体时，模型性能下降的更为明显。在 MQA 2 - HOP 任务中，Hit@1 下降至 98.9；MQA 3 - HOP 任务下，该值为 98.7。数据证明了验证智能体在整个智能体架构中的有效性。

在具有广泛代表性且数据特征较为复杂的 WebQSP 数据集下，其 F1 值达到 70.5，Hit@1 指标为 82。对模型模块进行消融，移除记忆模块后，模型性能出现了一定程度的下滑。WebQSP 数据集下，F1 值降至 70.2，Hit@1 降至 79.9。这清晰地表明记忆模块在模型运行过程中，承担着辅助智能体存储、调用以及关联关键信息的作用，其缺失使得模型在面对复杂问题时，信息处理的流畅性与准确性受到干扰。进一步地，当移除验证

智能体时,模型性能的退化更为明显。于 WebQSP 数据集而言, F1 值下降至 68.9, Hit@1 为 79.4。数据证明了验证智能体在整个智能体架构中的有效性。

表 3.4 消融实验(WebQSP)
Tab.3.4 Ablation experiments (WebQSP)

模型	F1	Hit@1
S-VEQA	70.5	82.0
-mem	70.2	79.9
-ver	68.9	79.4

综合对比这些变体模型与完整的 Self-VEQA Agent 的性能表现,可以得出结论:无论是记忆模块还是验证智能体,它们对于提升知识图谱问答(KGQA)性能都有着不可忽视的重要性。

尤为值得关注的是,通过深入分析不同数据集下的实验结果,发现验证模块对 Self-VEQA Agent 的影响在不同数据集之间存在显著差异。相较于 MetaQA 数据集,验证智能体在 WebQSP 数据集上的影响更为突出,主要是 WebQSP 数据集自身特性,该数据集中所蕴含的推理链相对更为复杂、更具挑战性。在处理 WebQSP 数据集中的问题时,由于问题的深度与广度往往更大,仅仅依靠问答智能体在首次尝试时就正确生成完整且精准的推理链变得异常困难。此时,验证智能体的介入能够对初步生成的推理链进行反复校验、修正与完善,引导智能体逐步接近最优答案,从而极大地提升最终的问答性能。

3.3.7 其他分析

(1) 交互轮数分析

表 3.5 交互轮数对性能的影响(WebQSP)
Tab.3.5 The impact of the number of iterations (WebQSP)

轮数	F1	Hit@1
轮数=1	69.8	80.4
轮数=2	70.5	82.0
轮数=3	70.1	81.1
轮数=4	69.8	80.4
轮数=5	63.9	74.3

考虑到研究过程中所涉及的两个数据集各自的数据量以及内在的复杂度差异,经过

权衡,最终选定 WebQSP 作为本实验极具代表性的数据集,以便能够更加精准、有效地呈现推理链数量对性能的影响规律。

在本研究中,深入探究了设置问答智能体(QA Agent)与验证智能体之间不同的交互轮数对 Self-VEQA Agent 性能所产生的影响。

在实验过程中,选取了具有一定典型性的 WebQSP 数据集作为主要研究对象。做出这一选择的原因在于,综合考量两个相关数据集的数据量以及内在复杂度,WebQSP 数据集能够较为全面且突出地反映出实际应用场景中的各类情况,具备充分的代表性。

通过对实验结果的细致分析,如表 3.5 交互轮数对性能的影响(WebQSP)所示,清晰地发现一个与直觉相悖的现象:增加交互轮数并未促使 Self-VEQA Agent 的性能得到提升,相反,在多数情形下,过多的交互轮数会引发性能的下滑。图 3.7 为迭代轮数对 WebQSP 数据集性能的影响,以折线图的方式呈现。具体而言,就 WebQSP 数据集而言,将交互轮数精准设置为 2 时,Self-VEQA Agent 展现出了最佳的性能表现。对于那些跳数相对较少的问题,在经过两轮交互之后,智能体往往已经能够充分捕捉到问题的关键信息,并依此推理得出正确结果。这是由于在首轮交互中,问答智能体能够基于问题迅速给出初步的答案候选项,而验证智能体则在第二轮交互时,对这些候选项进行判断,两轮紧密配合,使得信息的流转与处理达到最优。一旦交互轮数超出 2,例如提升至 3、4 甚至 5 轮时,一方面会使得整个交互过程变得冗长复杂,增加了时间成本与计算资源的消耗;另一方面,过多的信息反馈与迭代可能引入更多的噪声与不确定性,干扰智能体的判断,进而导致最终性能不升反降。

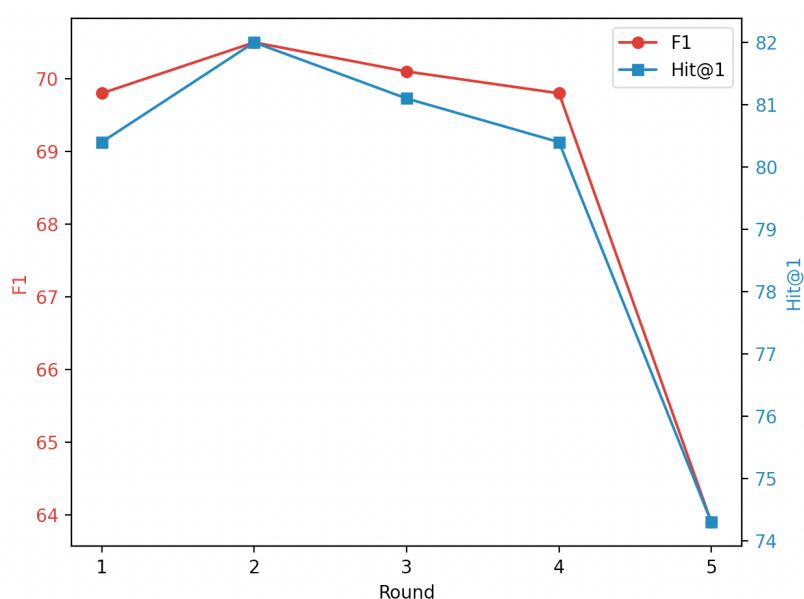


图 3.7 迭代轮数对性能的影响(WebQSP)

Fig. 3.7 The impact of the number of iterations (WebQSP)

详细的数据表现记录于表 3.5 中，该表清晰呈现了迭代轮数对 WebQSP 数据集性能的影响：当轮数为 1 时，F1 值达到 69.8，Hit@1 指标为 80.4；轮数设置为 2 时，F1 值提升至 70.5，Hit@1 指标也增长至 82.0，印证了此轮数下的性能优越性；轮数为 3 时，性能出现小幅度回落，F1 值降至 70.1，Hit@1 为 81.1；轮数为 4 时，各项指标进一步下滑，F1 值回到 69.8，Hit@1 降至 80.4，与轮数为 1 时相近；而当轮数高达 5 时，性能出现显著下降，F1 值下降至 63.9，Hit@1 也仅为 74.3。

综上所述，在设计与优化 Self-VEQA Agent 的交互机制时，并非交互轮数越多越好，而是应当依据数据集的特性以及问题的类型，精准地设定最为适宜的交互轮数，以此来实现智能体性能的最大化。

(2) 推理链数量分析

表 3.6 推理链的数量对性能的影响(WebQSP)

Tab.3.6 The impact of the number of inference chains(WebQSP)

推理链数量	F1	Hit@1
num=1	65.5	76.9
num=3	70.5	82.0
num=5	69.8	81.2
num=10	68.9	79.4

在本研究中，着重对设置问答智能体生成不同数量的推理链所带来的影响展开了深入分析。

实验表明，当把问答智能体生成的推理链数量精准设置为 3 时，整个模型展现出了最佳的性能表现。无论是在准确性、稳定性还是应对复杂任务的综合能力上，都达到了一个相对最优的状态。然而，当尝试增加生成的推理链数量时，情况却并不如预期那般理想。例如，将生成数量提升至 5 条，甚至进一步增加到 10 条时，整体任务性能不但没有提升，反而出现了明显的下降。这背后的原因在于，过多的推理链在验证智能体进行合理判断的环节中，引入了大量的噪声信息。这些多余的信息如同干扰项，使得验证智能体在筛选、甄别有效信息时面临挑战，进而严重影响了其判断的准确性与高效性。

与之相反，从实验数据也能清晰地观察到，当问答智能体生成的推理链数量过少时，同样会导致不理想的性能表现。过少的推理链意味着信息的挖掘与关联不够充分，无法全面、深入地应对复杂多变的任务需求，使得最终的结果缺乏足够的支撑与可靠性。

具体来看表 3.6 所展示的关于推理链的数量对 WebQSP 数据集性能的影响数据：当推理链数量设置为 1 时，WebQSP 数据集对应的 F1 值为 65.5，Hit@1 值达到了 76.9，

这表明在仅有一条推理链的情况下，虽然能够完成一定的任务，但在精度和召回率方面仍存在较大的提升空间；而当推理链数量为 3 时，F1 值上升至 70.5，Hit@1 值也提升到了 82.0；当推理链数量增加到 5 时，F1 值略微下降至 69.8，Hit@1 值为 81.2，性能下滑趋势初现；进一步将推理链数量扩展到 10 时，F1 值降至 68.9，Hit@1 值也降低到了 79.4，此时智能体性能受到的负面影响愈发显著。图 3.8 为推理链的数量对 WebQSP 数据集性能的影响，以折线图的方式呈现。

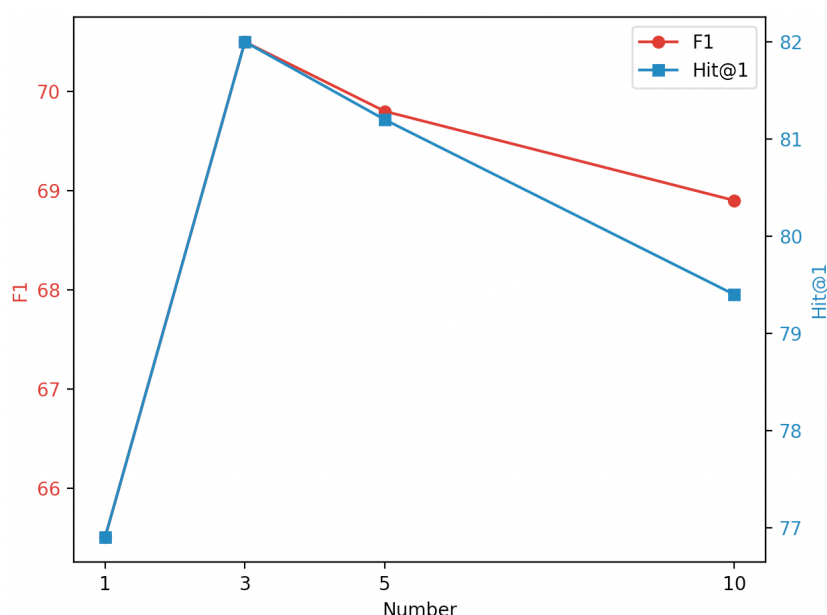


图 3.8 推理链的数量对性能的影响(WebQSP)

Fig. 3.8 The impact of the number of inference chains (WebQSP)

3.4 本章小结

本章重点介绍了 Self-VEQA Agent，旨在解决基于大语言模型的知识图谱问答中的多跳复杂问题。首先，本章阐述了多跳复杂问题求解的背景及其挑战，并概述了 Self-VEQA Agent 的整体架构及其运行机制。随后，详细解析了 Self-VEQA Agent 的核心模块，包括问答智能体和验证智能体，以及记忆模块，分别说明了它们在多跳推理中的作用和实现方式。最后，通过实验验证了各个模块的有效性，证明了 Self-VEQA Agent 在提升多跳复杂问题求解性能方面的显著优势。

4 基于迭代多跳知识检索与推理的图谱问答模型

4.1 引言

本章聚焦于基于信息检索的技术，旨在解决 KGQA 复杂多跳中的问题相关知识检索不全面的问题，进而提升 KGQA 任务的性能。在不进行微调的前提下，当前研究存在以下问题。首先，某些复杂问题的表述方式可能使大语言模型难以直接准确理解其语义。由于多跳问题通常涉及隐含的逻辑关系，大模型可能无法充分捕捉问题的核心意图，从而无法基于知识图谱进行有效的检索。这种理解上的偏差会进一步影响知识检索的准确性，导致检索模块难以获取与问题高度相关的知识片段，进而影响最终答案的质量和整体性能。

此外，Jiang^[4]等人利用检索知识图谱相关知识作为大语言模型外部知识补充的方法。该方法核心在于从海量知识中检索出与问题紧密关联的知识片段，将其作为大语言模型回答问题的外部知识源。然而，这些方法的检索模块直接基于原始问题进行检索，而原始问题通常是多跳的，涉及复杂的关系链和顺序依赖。由于检索模块难以准确捕捉多跳问题中隐含的关系顺序和逻辑正确性，导致检索到的知识片段不够全面或相关，从而影响最终答案的准确性。

最后，Kim^[37]等人通过问题分解解决多跳复杂问题，但方法存在局限。分解后的子问题可能质量不高，导致关系识别不准确，影响检索知识的全面性和相关性。例如，子问题语义不明确或逻辑关联不足，会使检索模块难以精准定位相关三元组，最终影响答案准确性。

针对上述挑战，本研究提出了 Iter-MKGRR。该模型通过多阶段的协同优化，显著提升了模型在复杂问答任务中的性能。首先，针对复杂问题表述导致大模型难以准确理解语义的问题，本研究引入语义扩充技术，利用 prompt 工程对原始问题进行深度语义增强，以提升模型对问题意图的精准捕捉能力；其次，为了在知识图谱中高效提取与子问题更相关和更全面的三元组，受 Wei^[17]等人的启发，本文引入了问题分解的方式对检索模块进行改进。采用问题分解策略，将复杂的多跳问题拆解为一系列语义明确的单跳子问题，并基于子问题迭代检索知识图谱，从而解决检索模块中知识不全面或相关性不足的问题。为进一步解决子问题关系识别不准确的问题，本文提出知识整合技术，通过将子问题的答案与当前问题进行动态融合，构建连贯且逻辑清晰的上下文信息，以增强模型的语义理解能力。最终，依托大语言模型的深度推理能力，结合检索到的三元组与模型内部的知识储备，生成最终答案。

4.2 方法

4.2.1 模型概述

Iter-MKGRR 主要分为四个模块，分别为语义扩充模块，问题分解模块，知识图谱检索模块和大语言模型推理模块，可见图 4.1。通过语义扩充技术增强问题表述的清晰性，利用问题分解策略将复杂多跳问题拆解为单跳子问题，结合迭代知识图谱检索机制精准定位相关三元组，并引入知识整合技术动态融合子问题答案，构建连贯的上下文信息。最终，依托大语言模型的推理能力，结合检索知识与内部知识生成准确答案。该框架显著提升了复杂多跳问答任务的性能，为 KGQA 领域提供了高效、可靠的解决方案。模型的具体算法流程见图 4.2。

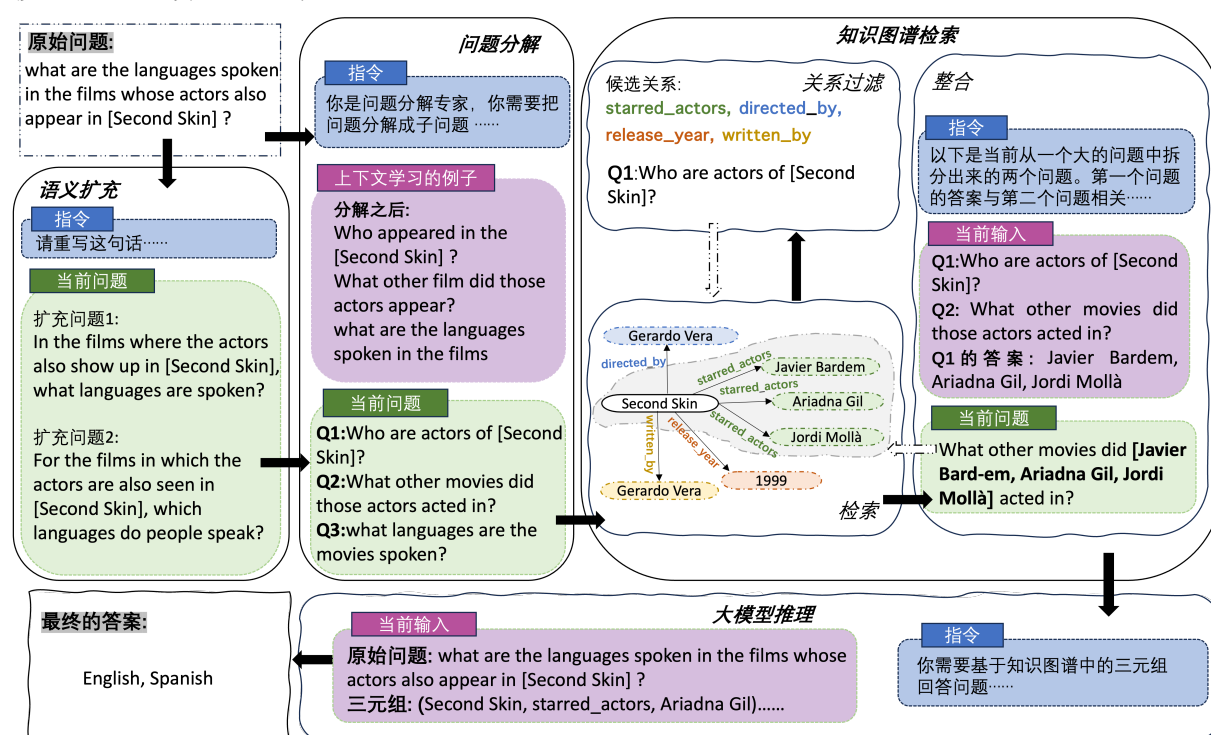


图 4.1 Iter-MKGRR 的整体框架

Fig. 4.1 Structure of Iter-MKGRR

4.2.2 语义扩充

在当前这一关键部分的研究工作中，挑选了 OpenAI 所推出的 GPT-3.5-turbo 作为基础大模型。之所以做出这样的选择，是因为 GPT-3.5-turbo 在自然语言处理领域展现出了强大的语言理解与生成能力，拥有庞大的知识储备和良好的泛化性能，为本研究提供了坚实的基础。

算法4.1: Iter-KGRR算法

输入: 原始问题 $Q_{original}$, 主题实体 $head$, 知识图谱 KG
 输出: 最终答案 A_{final}

- 1: 初始化大模型 M 为 *GPT 3.5 Turbo*
- 2: $Q_{expanded} \leftarrow$ 语义扩充($Q_{original}, M$)
- 3: $S_{subquestions} \leftarrow$ 问题分解($Q_{original} + Q_{expanded}, M$)
- 4: 初始化所有相关三元组集合 T_{all} 为空
- 5: 对于每个子问题 Q_{sub} 在 $S_{subquestions}$ 中做
- 6: $R_{current} \leftarrow$ 关系过滤($head / A_{prev}, Q_{subrelation}, M$)
- 7: $A_{current}, T_{relevant} \leftarrow$ 答案检索($R_{current}, KG, head / A_{prev}$)
- 8: if $A_{current}$ 不为空 then
- 9: $Q_{sub} \leftarrow$ 知识整合($A_{current}$, 当前子问题, $Q_{subnext}$)
- 10: 将 $T_{relevant}$ 添加到 T_{all} 中
- 11: end if
- 12: 结束循环
- 13: $A_{final} \leftarrow$ 大模型推理(T_{all}, M)
- 14: 返回 A_{final}

图 4.2 Iter-MKGRR 算法流程

Fig. 4.2 Iter-MKGRR Algorithm Process

为了充分挖掘该模型的潜力, 使其在知识图谱问答(KGQA)任务中发挥出更出色的表现, 本研究引入了 **prompt** 工程这一重要技术手段。**prompt** 工程不仅仅是简单的文本提示, 它是一种通过巧妙设计输入文本, 引导模型生成更符合预期结果的方法。在该研究里, 运用 **prompt** 工程对原始问题进行语义上的扩充。这一过程并非随意添加信息, 而是基于大模型对问题的深入分析理解, 有针对性地对原问题进行丰富和拓展。

之所以要进行这样的操作, 是因为大量的实验已经明确证明, 问题的呈现形式会影响大模型对问题的理解。当问题的语义不够清晰、表达不够完整时, 大模型可能会误解问题的含义, 从而影响后续的流程。而经过 **prompt** 工程处理后的问题, 由于语义得到了扩充和优化, 大模型能够更准确从不同角度地把握问题的核心, 进而在后续问题分解任务中, 会具有更稳定的性能。

4.2.3 问题分解

在知识图谱问答领域, 多跳问题始终是极具挑战性的难题。多跳问题往往涉及多个实体和复杂的关系链, 需要模型在知识图谱中进行多次跳转和推理才能找到答案, 这对模型的推理能力和知识理解能力提出了极高的要求。为了攻克这一难题, 研究人员提出了诸如思维链(Chain of Thought, COT)等方法。这些方法旨在引导模型逐步推理, 通过中间步骤来解决复杂的多跳问题。大量实验结果表明, 大语言模型在处理单跳简单问题

时的表现要显著优于多跳复杂问题。对于单跳问题，模型通常可以直接根据问题中的实体和关系，基于知识图谱中的三元组快速定位到答案。然而，面对多跳问题，模型需要在多个实体和关系之间进行复杂的推理和跳转，这容易导致推理错误或者无法找到正确的答案路径。

问题分解

*You are an expert in question decomposition.
 You can break down long questions into 3 sub-questions when it is necessary.
 You need to understand the question.
 Here is the target original question you need to decompose:{question}
 Note:
 The first subquestion is always related to contents in []
 The final sub-question is always related to what the original question actually wants to ask.
 You only need to directly output three subquestions without label 'output:' or 'Subquestions:'.
 There is no necessary to repeat last one subquestion.
 There is no need to output original question.
 Make sure that every question has only one verb.
 Here are some examples you can refer to.*

Example Original question:
When did the movies release whose writers also wrote [Parineeta]?

Output:
*Who are the writers of [Parineeta]?
 What other movies did those writers write?
 When were those movies released?*

Example Original question:
the films that share directors with the film [Catch Me If You Can] were in which languages

Output:
*What is the director of the film [Catch Me If You Can]?
 Which films is directed by the directors?
 What languages are the movies?
*

图 4.3 问题分解的 prompt

Fig.4.3 Prompt of question decomposition

在本篇论文中，采用了对冗长且困难的多跳问题进行逐步分解方法。核心思路是将复杂的多跳问题拆解为一系列大语言模型表现出色的单跳问题。给定一个复杂的多跳问题 Q ，问题分解的目标是将其拆解为一系列语义明确的单跳子问题 $\{q_1, q_2, \dots, q_n\}$ ，其中每个子问题 q_i ，仅涉及单一的知识图谱查询或推理步骤。通过这种方式，将原本复杂的推理过程转化为多个简单的推理步骤，降低了问题的难度，提高了模型解决问题的准确性和效率。在具体实现过程中，本研究选择了 OpenAI 的 GPT-3.5-turbo 作为基础大模型。GPT-3.5-turbo 凭借其强大的语言理解和生成能力，为问题分解任务提供了基础。为了实现对原问题的有效分解，本研究也是通过提示工程技术实现。具体的见图 4.3，因为主要使用的是 OpenAI 的 GPT 系列，所以 prompt 是英文。

为了进一步提升问题分解的性能，本研究沿用了第三章所提及方法中的跳数预测部分。跳数预测可以帮助预先了解问题的复杂程度，明确需要分解的步骤数量。通过准确

预测跳数，能够更有针对性地设计提示，使模型在分解问题时更加高效和准确。例如，如果预测到问题为三跳问题，可以引导模型将其分解为三个单跳问题，并确保每个单跳问题之间的逻辑关系清晰，从而为后续的答案推理奠定良好的基础。

4.2.4 知识图谱检索

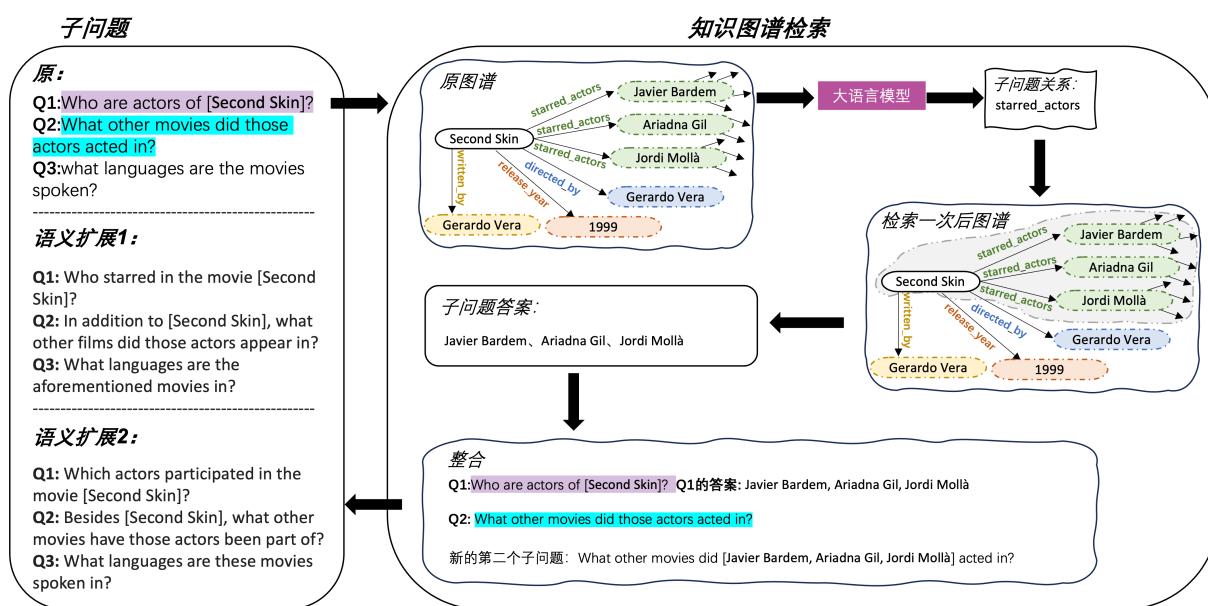


图 4.4 知识检索详细过程

Fig.4.4 The detailed process of knowledge retrieval

在知识图谱问答任务中，从知识图谱里检索三元组这一环节起着至关重要的作用，它是获取准确答案的关键步骤。这一阶段的核心任务是针对问题分解后得到的每一个子问题，在知识图谱中检索与之相关的三元组，并找出可能的候选答案。只有准确地检索到相关三元组，才能为后续的答案推理和生成提供坚实的基础。下面是具体的步骤。

首先，会获取问题主题实体的一阶邻域三元组。一阶邻域三元组包含了与主题实体直接相连的所有关系和实体信息，这些信息为进一步探索知识图谱提供了丰富的信息。通过获取这些三元组，能够全面了解当前主题实体所涉及的所有关系，为后续的筛选和推理做好准备。为了提高检索的效率和准确性，选用了 OpenAI 的 GPT-3.5-turbo 作为基础大模型，并运用了提示工程技术。在本研究中，利用提示工程对获取到的关系进行筛选。向 GPT-3.5-turbo 输入包含问题信息和关系列表的提示，引导它根据问题的语义和知识图谱的逻辑，从众多关系中筛选出最相关的前 k 个关系(即 top k 关系)。得到 top k 关系后，会依据这些关系对主题实体的一阶邻域三元组进行再次筛选。通过这种筛选，能够排除那些与问题不太相关的三元组，只保留与 top k 关系匹配的三元组。经过

这一步骤，得到的尾实体将被作为新的主题实体。随后，会重复上述过程，不断获取新主题实体的一阶邻域三元组，进行关系筛选和三元组筛选，直到满足特定的循环停止条件。这个循环停止条件可以是达到了预设的跳数、检索到了足够的候选答案或者没有更多相关的三元组可供检索等。通过这种逐步深入、循环检索的方式，能够在知识图谱中全面、准确地检索到与每个子问题相关的三元组和候选答案，为后续的答案生成和验证提供了丰富且可靠的信息资源，从而提高知识图谱问答任务的整体性能和准确性。具体可见图 4.4。

4.2.5 知识整合

知识整合

*There are currently two questions split from a big question.
The answer to the first question is relevant to the second question.
It is necessary to replace part of the second question with the answer to the first question.
I will provide you with the answer to the first question, and you need to do the corresponding sorting.
Here is the question1:{question1}.
Here is the answer1:{answer}.
Here is the question2:{question2}.
Here is the examples:*

example1:
question1:*Who are the writers of [Parineeta]?*
answer1:*['Vidhu Vinod Chopra', 'Pradeep Sarkar']*
question2:*What other movies did those writers write?*
changed_question2:*What other movies did [Vidhu Vinod Chopra][Pradeep Sarkar] write?*

*Note: You only need to output the question2 you've changed based on the answer to the question1.
Note that you must not change the original semantics of question2. You can only make replacements.
Note that you must ensure content you put in [] should not contain extra symbols including single/double quotation marks like changed_question2 of example1.
You need to ensure that there is no nested form of [[]].
If there are more than one element in answer1, you should separate them, like [element1][elements].
You need to ensure that every element in answer is in its [].*

图 4.5 知识整合部分的 prompt

Fig.4.5 Prompt of Knowledge Integration

在知识检索工作里，当成功获取到前一个步骤所得到的答案后，一项至关重要的操作就是将上一步得出的答案以及上一个子问题与当前子问题进行处理，这种处理方式既可以看作是替换，也能理解为整合。通过大量的实验验证，发现经过整合之后的问题，能够为大语言模型提供更加清晰、连贯且富有逻辑的信息，从而极大地有助于大语言模型对问题进行理解，进而更高效、准确地进行知识检索和答案生成。然而，若不将这些

答案与当前问题结合起来，它们可能只是孤立的数据片段，无法形成连贯且有意义的整体。

具体而言，在问题分解中，将一个复杂的大问题拆分成多个具有逻辑关联的子问题。此时，往往会出现一种情况，即第一个子问题的答案与第二个子问题紧密相关。设原始问题 Q 被分解为一系列逻辑相关的子问题 $\{q_1, q_2, \dots, q_n\}$ ，其中每个子问题 q_i 依赖于前一个子问题 q_{i-1} 的答案。对于第 i 个子问题 q_i ，其输入为前一个子问题 q_{i-1} 的答案 a_{i-1} 。通过将 a_{i-1} 整合到 q_i 中，生成一个新的明确子问题 q'_i ，形式化表示为：

$$q'_i = \text{Replace}(q_i, \text{Reference}(q_i), a_{i-1}) \quad (4.1)$$

其中， $\text{Reference}(q_i)$ 表示 q_i 中对前一个答案的引用部分(如“那些演员”)。 Replace 表示将引用部分替换为具体的答案 a_{i-1} 。整合后的子问题 q'_i 被输入到大语言模型中，用于知识检索或推理，生成答案 a_i 。具体的见图 4.5。

下面通过一个具体的例子来详细说明这个过程。假设最初有一个复杂的大问题，经过合理的拆分后，得到了两个子问题。第一个子问题 Q_1 是“电影《第二层皮肤》的演员都有谁？”，第二个子问题 Q_2 是“这些演员还出演过其他哪些电影？”。经过知识检索，得到了针对 Q_1 的答案，即“Javier Bardem、Ariadna Gil、Jordi Mollà”。接下来，要进行前面所说的整合操作。具体做法是，把 Q_1 的答案代入到 Q_2 中，替换掉 Q_2 里指代不明确的“这些演员”。经过这样的整合，原本的 Q_2 就变成了“[Javier Bardem][Ariadna Gil][Jordi Mollà]还出演过其他哪些电影？”。通过这样的整合，大语言模型在接收到这个问题时，无需再去额外推断“这些演员”具体是谁，能够直接依据明确的演员信息进行下一步的推理，大大提高了处理问题的效率和准确性。

4.2.6 大模型推理

在历经前面多个步骤的反复迭代之后，所构建的模型成功检索到了足以解决初始问题的丰富知识。如今，模型进入到至关重要的推理阶段，在这个阶段，大语言模型将充分利用前期所获取的这些知识进行推理。

具体而言，大语言模型会把检索得到的相关三元组当作推理的核心知识源。这些三元组具备标准的格式，即[头实体，关系，尾实体]，它们清晰地呈现了知识图谱中不同实体之间的关联。同时，大语言模型并不会仅仅依赖于这些外部检索到的三元组，它还会结合自身的内部知识，也就是在大规模数据上预训练所积累的丰富语义理解和逻辑推理能力。大语言模型会对这些知识进行整合、推导和判断，逐步从已知信息推导出未知的结论，最终得出能够有效解决初始问题的答案。

4.2.7 案例分析



图 4.6 案例分析

Fig.4.6 Case study of Iter-MKGRR

图 4.6 为 Iter-MKGRR 的一个具体示例。针对问题“Which person wrote the films acted by [Torrey DeVitto]?”，首先通过问题分解将其拆解为两个子问题：['What films are acted by [Torrey DeVitto?]', 'Which person wrote those films?']。在知识检索过程中，首先处理第一个子问题“Which person wrote the films acted by [Torrey DeVitto]?”，识别主题实体“Torrey DeVitto”，并通过大模型过滤与主题实体相关的关系，得到三元组[Killer Movie starred_actors Torrey DeVitto]，从而确定当前子问题的答案为“Killer Movie”。在此基础上，通过问题整合将答案与第二个子问题结合，生成新的问题“Which person wrote [Killer Movie]?”，并进一步检索得到三元组[Killer Movie written_by Jeff Fisher]。最终，基于检索到的全部三元组[Killer Movie starred_actors Torrey DeVitto]和[Killer Movie written_by Jeff Fisher]进行推理，生成最终答案“Jeff Fisher”。这一过程展示了模型通过问题分解、知识检索与整合，逐步解决复杂多跳问题的能力。语义扩展后和原问题走同样的流程，进而丰富了大模型推理时候用的三元组，所以语义扩展的重复步骤就不在图中体现。

4.3 实验

4.3.1 数据集

本研究在具有代表性的小规模和大规模知识图谱及其对应的知识图谱问答(KGQA)数据集上进行了评估。

(1) MetaQA^[63]数据集

MetaQA 是电影领域的一个大型 KGQA 数据集，其知识图谱包含 43000 个实体、9 种关系和 135000 个三元组。它包含 407000 个需要从头部实体进行 1 跳至 3 跳推理的问题。每个问题由一个头部实体、一个关系推理路径和答案实体组成。为了评估 Iter-MKGR 的多跳推理能力，在实验中重点关注 MetaQA 中的 2 跳和 3 跳数据集。

(2) WebQSP 数据集

WebQSP 这是一个问题数量较少，但知识图谱规模较大的基准数据集。它包含 Freebase^[66]上最多 2 跳的问题，每个问题都有一个主题实体、约束条件、推理链和用于查找答案的 SPARQL 查询。本研究使用了谷歌提供的最新 Freebase 数据转储，截至 2023 年，其中包含 31.2 亿个三元组。WebQSP 有 4737 个问题，但排除了 11 个没有标准答案的问题。

4.3.2 评估指标

与先前的研究一致，本研究采用 Hits@1 和 F1 作为评估指标。Hits@1 衡量的是预测的排名第一的答案正确的问题所占的百分比，能够直观反映模型在单答案场景下的准确性。F1 指标则考虑了多答案场景下的性能，通过平衡精确率(Precision)和召回率(Recall)来评估模型的综合表现。精确率衡量预测答案中正确结果的比例，而召回率则衡量模型能够覆盖多少标准答案。

4.3.3 基线模型

为了评估 Iter-MKGR 的有效性，本研究在 WebQSP 和 MetaQA 数据集上，将其与知识图谱问答领域中一系列已有的基线模型进行了对比分析。这些基线模型可以分为不包含大语言模型(LLM)的传统方法和基于大语言模型的方法。

(1) 传统方法基线

包括 KV-Mem^[1]、EmbedKGQA^[2]。所有这些基线模型都在 MetaQA 和 WebQSP 数据集上进行了评估。

(2) 基于大语言模型的基线

本研究添加了 StructGPT^[4]、KnowledgeNavigator^[36]、TOG^[27]、ReasoningLM^[52]等作

为利用大语言模型的知识图谱问答任务的基线模型。

4.3.4 实现细节

本研究通过 OpenAI API 使用闭源的 GPT-3.5-turbo 模型。为了保证结果可复现，温度参数设置为 0.0。在基础实验中，语义扩充的问题数量为两个。关系过滤选取最相关的一个关系。

4.3.5 主要结果与分析

表 4.1 Hit@1 结果(MetaQA)
Tab.4.1 Hit@1 results (MetaQA)

方法名	MQA 2-HOP	MQA 3-HOP	Δ
KV-Mem ^[1]	82.7	48.9	-33.8
EmbedKGQA ^[2]	98.8	94.8	-4.0
Davinci-003 ^[56]	25.3	42.5	-12.2
ChatGPT	31.0	43.2	-12.2
GPT4	52.5	49.2	-3.3
StructGPT(ChatGPT) ^[4]	97.3	87.0	-10.3
KnowledgeNavigator ^[36]	99.5	95.0	-4.5
ReasoningLM ^[52]	98.3	92.7	-5.6
Iter-MKGRR(Ours-GPT)	99.0	96.0	-3.0
Iter-MKGRR(Ours-GPT*)	99.2	97.7	-1.5
Iter-MKGRR(Ours-Deepseek)	98.6	96.6	-2.0
Iter-MKGRR(Ours-Deepseek*)	<u>99.5</u>	<u>99.0</u>	<u>-0.5</u>

(1) 与传统预训练语言模型对比与分析

在本研究中，KV-Mem、EmbedKGQA 为传统预训练语言模型。如表 4.1 所示，最佳结果以粗体显示。 Δ 表示在 MetaQA 上从两跳任务到三跳任务的性能下降情况。KV-Mem 的 82.7 相对较低；而在 MQA 3-HOP 任务中，KV-Mem 仅为 48.9，且从 MQA 2-HOP 到 MQA 3-HOP 的变化(Δ Hit@1)中，KV-Mem 下降幅度最大，达到 -33.8。虽然传统模型在 MetaQA 上性能较好，然而，当面对更为复杂的 WebQSP 数据集时，情况发生了变化，如表 4.2 所示。其他三种传统模型，与 Iter-MKGRR 相比，整体仍存在差距。Iter-MKGRR 在 MQA 2-HOP 的 Hit@1 为 99.0，MQA 3-HOP 为 96.0，在 WebQSP 数据

集上, F1 值为 74.5、Hit@1 值为 80.2, 相比之下, 传统模型与 Iter-MKGRR 的差距显著。

表 4.2 F1 和 Hit@1 结果(WebQSP)
Tab.4.2 The results of F1 and Hit@1(WebQSP)

方法名	F1	Hit@1
KV-Mem ^[1]	34.5	46.7
EmbedKGQA ^[2]	-	66.6
TOG(llama) ^[27]	-	68.9
Davinci-003 ^[56]	-	48.7
ChatGPT	-	61.2
GPT4	-	70.7
StructGPT(ChatGPT) ^[4]	-	72.6
KnowledgeNavigator ^[36]	-	83.5
RoG ^[6]	70.8	85.7
ReasoningLM ^[52]	71.0	78.5
Iter-MKGRR(Ours)	74.5	80.2
Iter-MKGRR(Ours-GPT*)	75.0	80.8
Iter-MKGRR(Ours-Deepseek)	74.0	85.1
Iter-MKGRR(Ours-Deepseek*)	<u>75.1</u>	<u>86.3</u>

通过深入分析可以发现, 传统预训练语言模型, 包括 KV-Mem、EmbedKGQA 在不同数据集上的性能差异主要源于数据集特性的显著不同。具体而言, MetaQA 数据集的结构相对简单直接, 其答案通常可以通过完整的关系链直接推导得出; 而 WebQSP 数据集则具有更高的复杂性, 需要在多重约束条件下对关系进行精细化筛选和推理, 才能准确定位答案。这一对比清晰地表明, 尽管传统预训练语言模型已经初步具备了基础的语言理解能力, 但在处理具有复杂约束条件的推理任务时, 其性能仍存在明显的局限性, 需进一步优化和提升。

传统预训练语言模型无法像 Iter-MKGRR 模型那样高效地解构复杂问题并深入挖掘其中隐含的语义信息。具体而言, 传统模型在问题理解阶段往往局限于表层语义, 难以对复杂问题进行多层次的分解和语义扩展, 这一缺陷直接影响了后续的知识图谱检索环节。由于检索过程缺乏对问题的精细解析, 传统模型难以准确地将问题语义与图谱中的知识节点进行匹配, 从而导致检索精度下降, 并最终显著降低了问答的整体准确率。

以 KV-Mem 模型为例,该模型借鉴了传统键值记忆神经网络的设计思路,创新性地将答案-问题对建模为键值对,并基于此模式进行训练。在面对复杂问题时, KV-Mem 能够生成具有一定可解释性的推理过程,这一特性为其赋予了初步的推理能力。然而,该模型的推理机制存在明显的局限性:其推理过程相对刻板且缺乏灵活性,无法根据问题的语义特征进行动态调整或深度拆解。特别是在面对表述多样、语义模糊的问题时, KV-Mem 难以像 Iter-MKGRR 模型那样,通过语义扩展功能灵活适应不同的问法,也无法利用问题分解策略为知识图谱检索提供精准的指引。这种固有的局限性导致 KV-Mem 在复杂应用场景中的表现不尽如人意,难以满足实际应用中对高精度问答的需求。这一案例进一步印证了语义扩展和问题分解模块在提升知识图谱问答系统性能中的关键作用。

(2) 基于大模型的知识问答方法对比与分析

基于大模型的知识问答方法为表 4.1 和 4.2 中 TOG(llama)、Davinci-003、ChatGPT、GPT4、StructGPT(ChatGPT)、KnowledgeNavigator、RoG 和 ReasoningLM。以 KnowledgeNavigator 为代表的先进方法在部分任务上取得了显著成果,例如在 MetaQA 2-HOP 数据集上达到了 99.5 的准确率。然而,在更具挑战性的 MetaQA 3-HOP 任务中,其性能却不及 Iter-MKGRR 框架。主要是由于 Iter-MKGRR 设计了更加详细的知识整合,这有助于大语言模型更准确的抽取关系链。与 ROG 方法比,ROG 主要的优势在于其对大语言模型使用指令微调,然而在 WebQSP 数据集上 F1 指标上,本文的方法是优于 ROG 的。与纯大模型方法(如 ChatGPT、GPT-4、Davinci-003)相比,Iter-MKGRR 展现出了独特的优势。尽管 Iter-MKGRR 同样依托于大模型强大的推理能力,但其引入了问题分解和语义扩展模块,从而显著提升了复杂问题的处理能力。具体而言,当面对表述模糊、语义复杂的难题时,Iter-MKGRR 能够通过预处理机制将问题拆解为多个清晰的子问题,并扩展语义边界以挖掘潜在信息。这种设计为后续的知识图谱检索和大模型推理提供了更精准的输入,全面增强了系统的鲁棒性,有效避免了因问题理解偏差导致的错误。这一能力正是纯大模型所欠缺的。

此外,Iter-MKGRR 的另一个显著优势在于其无需对大模型进行额外的训练或微调,这不仅大幅降低了成本和时间开销,还使其在知识问答任务中保持了一定的性能。

(3) 多跳推理场景下的方法性能对比与分析

由表 4.1 中的实验结果表明,随着问题跳数的增加,大多数方法的性能均出现显著下降,而 Iter-MKGRR 模型的性能下降幅度相对较小,展现出更强的稳定性。这一优势主要归功于 Iter-MKGRR 模型中的设计。具体而言,在面对需要多跳推理的复杂问题时,Iter-MKGRR 的问题分解模块能够将原始问题拆解为多个清晰的子问题,使每一跳的检

索目标更加明确；语义扩展模块则通过多维度语义理解，有效避免了多跳路径探索中的语义偏差；同时，图谱检索与大模型推理的协同机制确保了在多跳过程中获取的大量信息能够被精准筛选和整合，从而得出最符合逻辑的答案。相比之下，其他方法在二跳和三跳任务中性能提升有限，甚至出现性能瓶颈，而 Iter-MKGRR 能够维持较高的准确率，进一步凸显了其在复杂知识图谱问答场景下处理多跳推理任务的卓越能力。

在多跳推理场景中，Iter-MKGRR 表现出色且性能稳定。从 $\Delta\text{Hit}@1$ 指标来看，Iter-MKGRR 的下降幅度仅为-3.0，显著低于其他方法，这表明其在多跳任务转换过程中性能波动较小。与其他方法相比，Iter-MKGRR 能够更好地应对多跳推理带来的信息复杂性，通过模块间的高效协作，精准地获取和推理多跳关系中的答案。这一特性充分体现了 Iter-MKGRR 在复杂知识图谱问答场景下的技术优势，为多跳推理任务提供了一种高效且可靠的解决方案。

(4) GPT 与 DeepSeek 基座模型的对比与分析

本研究使用 DeepSeek V3 作为基座模型进行了实验，实验结果如表 4.1 和 4.2 所示，其中使用下划线标注出的为最高性能。其中 Ours-DeepSeek*表示由于涉及国家安全的信息、政治与宗教类的信息等原因，调用 DeepSeek API 时无法回答的数据。在对比实验中，使用 DeepSeek V3 的 Iter-MKGRR 模型在性能上表现优于 GPT-3.5-Turbo，这一性能提升可归因于 DeepSeek V3 在逻辑分解能力上的优化。相较于 GPT，DeepSeek V3 通过升级模型架构，如增强的注意力机制或更深的网络层，进一步强化了对复杂任务的层次化推理能力，使其能够将多步骤问题拆解为更清晰的子任务序列以及后续的知识整合任务，从而更精准地匹配对结构化输出与多步逻辑链的要求，进而提升整体的性能。

4.3.6 消融实验

本研究对 Iter-MKGRR 模型进行消融实验，如表 4.3 和表 4.4 所示，旨在探究语义扩充、问题分解和语义整合等模块对模型性能的影响。其中 Iter-MKGRR 表示的是本论文中提出的模型。实验设置了三个变体模型：去除语义扩充的-seman 模型、去除问题分解模块的-decom 模型以及去除语义整合的-compose 模型，并在 MQA 2-HOP、MQA 3-HOP 和 WebQSP 数据集上进行评估。其中-compose 主要使用问题分解的子问题直接生成推理链。-decom，模型使用原问题直接生成推理链。

对于 MQA 2-HOP 数据集，Iter-MKGRR 模型的 Hit@1 为 99.0。去除语义扩充的-seman 模型，Hit@1 降至 97.0，说明语义扩充模块有助于提升模型在此任务上的准确性。去除问题分解模块的-decom 模型，Hit@1 大幅下降至 85.0，表明问题分解模块在二跳推理任务中对模型性能至关重要。去除语义整合的-compose 模型，Hit@1 为 92.0，说明语义整合模块也对模型性能有一定贡献。

表 4.3 消融实验(MetaQA)

Tab.4.3 Ablation experiments (MetaQA)

模型	MQA 2-HOP	MQA 3-HOP
Iter-MKGRR	99.0	96.0
-seman	97.0	94.4
-decom	85.0	80.0
-compose	92.0	89.0

对于 MQA 3-HOP 数据集,Iter-MKGRR 模型的 Hit@1 为 96.0。-seman 模型的 Hit@1 为 94.4, 再次体现语义扩充模块的作用。-decom 模型的 Hit@1 为 80.0, 远低于 Iter-MKGRR 模型, 进一步证明问题分解模块在三跳推理任务中的重要性。-compose 模型的 Hit@1 为 89.0, 表明语义整合模块对模型在三跳任务中的性能也有影响。

对于 WebQSP 数据集, 如表 4.4 所示, Iter-MKGRR 模型的 F1 值为 74.5, Hit@1 为 80.2%。-seman 模型的 F1 值降至 70.1, Hit@1 降至 78.9, 凸显语义扩充模块在处理复杂任务时的重要性。因为该数据集的语义特殊性, 并没有对其进行问题分解, 所以没有做这部分的消融实验。

表 4.4 消融实验(WebQSP)

Tab.4.4 Ablation experiments (WebQSP)

模型	F1	Hit@1
Iter-MKGRR	74.5	80.2
-seman	70.1	78.9

综合来看, 语义扩充、问题分解和语义整合模块均对 Iter-MKGRR 模型的性能有重要影响。语义扩充模块在不同数据集和任务中均能提升模型性能, 问题分解模块在多跳推理任务中作用关键, 语义整合模块也在复杂任务场景中对模型性能有显著贡献。这些发现为 Iter-MKGRR 模型的进一步优化和改进提供了明确的方向。

4.3.7 其他分析

(1) 语义扩充数量对性能的影响分析实验

MQA 2-HOP 数据集上, 如表 4.5 所示, 当语义扩充数量为 0 时, Hit@1 为 92.0; 数量增加到 1 时, Hit@1 提升至 97.0; 扩充数量为 2 时, Hit@1 达到 99.0, 为该数据集

下的最高值；而当数量为 3 时，Hit@1 又降回 99.0。这表明在 MQA 2-HOP 任务中，适度的语义扩充(数量为 1 和 2 时)有助于提升模型性能，扩充数量为 2 时效果最佳，过多的语义扩充(数量为 3 时)可能会导致性能下降。

表 4.5 语义扩充数量对 Hit@1 的影响(MetaQA)

Tab.4.5 The influence of the quantity of semantic expansion (MetaQA)

数量	MQA 2-HOP	MQA 3-HOP
0	92.0	89.0
1	97.0	94.4
2	99.0	96.0
3	98.0	95.7

MQA 3-HOP 数据集，如表 4.5 所示，语义扩充数量为 0 时，Hit@1 是 89.0；数量为 1 时，Hit@1 升至 94.4；数量为 2 时，Hit@1 进一步提高到 96.0；数量为 3 时，Hit@1 又回到 95.7。与 MQA 2-HOP 数据集类似，在 MQA 3-HOP 任务中，适量的语义扩充(数量为 1 和 2 时)能提升模型性能，扩充数量为 2 时效果较好，过多的扩充则使性能回落。

表 4.6 语义扩充数量对性能的影响(WebQSP)

Tab.4.6 The influence of the quantity of semantic expansion (WebQSP)

数量	F1	Hit@1
0	62.1	70.9
1	70.1	78.9
2	74.5	82.0
3	74.4	80.2

WebQSP 数据集，如表 4.6 所示，语义扩充数量为 0 时，F1 值为 62.1，Hit@1 为 70.9%；数量为 1 时，F1 值提升到 70.1，Hit@1 升至 78.9；数量为 2 时，F1 值达到 74.5，Hit@1 为 82.0。数量为 3 时，Hit@1 和 F1 值也会稍稍下降。这可能是由于检索轮数变多或太少，三元组数量太多或太少，也会在一定程度上影响性能。

综合来看，在 MQA 2-HOP、MQA 3-HOP 和 WebQSP 数据集中，适度的语义扩充能提升模型性能，过多则可能导致性能下降。这为确定合适的语义扩充数量以优化模型性能提供了参考依据。

(2) 推理大模型对性能的影响实验分析

本次实验围绕 GPT-3.5-turbo 与 GPT-4 这两种推理大模型展开，旨在深入剖析它们使用不同的大模型对其性能的影响，具体任务涵盖 MQA 2-HOP、MQA 3-HOP 以及 WebQSP，评估维度聚焦于指标 Hit@1 与 WebQSP 的 F1 值。

表 4.7 推理大模型对 Hit@1 的影响(MetaQA)

Tab.4.7 The influence of different large models on reasoning performance (MetaQA)

模型名	MQA 2-HOP	MQA 3-HOP
GPT-3.5 turbo	95.0	88.0
GPT-4	99.0	96.0

在 MQA 2-HOP 任务中，如表 4.7 所示，GPT-4 以 99.0 的 Hit@1 显著优于 GPT-3.5-turbo 的 95.0。当任务复杂度提升至 MQA 3-HOP 时，GPT-4 的优势进一步凸显，其 Hit@1 达到 96.0，而 GPT-3.5-turbo 仅为 88.0。这一显著差距表明，随着推理链路的延长和复杂度的增加，GPT-4 有更好的性能。

表 4.8 推理大模型对 Hit@1 和 F1 的影响(WebQSP)

Tab.4.8 The influence of different large models on reasoning performance (WebQSP)

模型名	F1	Hit@1
GPT-3.5 turbo	68.1	76.0
GPT-4	74.5	80.2

在模拟真实世界复杂问答场景的 WebQSP 任务中，如表 4.8 所示，GPT-4 以 74.5 的 F1 值和 80.2 的 Hit@1 全面超越 GPT-3.5-turbo 的 68.1(F1)和 76.0(Hit@1)。

综合来看，参数量更多更复杂的 GPT-4 作为推理大模型有更好的性能。

4.4 本章小结

在本研究中，提出了 Iter-MKGRR，旨在解决基于大语言模型的知识图谱问答任务中的多跳复杂问题。该框架将大语言模型引进 KGQA 任务。通过将外部知识图谱的检索结果与大模型的内部知识相结合，该框架能够更有效地处理复杂的多跳问题，显著提升了问答的准确性。具体而言，Iter-MKGRR 框架通过语义扩充技术增强问题理解能力，采用问题分解策略降低推理复杂度，结合知识图谱检索机制高效提取相关三元组，并运用知识整合技术构建连贯的上下文信息，最终依托大语言模型的推理能力生成准确答案。

实验结果表明，该框架在处理复杂多跳问题时展现了更强的准确性，为知识图谱问答领域提供了更加高效、可靠的解决方案。

在技术实现上，本研究通过精心设计的 **prompt** 工程，引导大语言模型更准确地理解问题的语义，从而在问题分解和知识检索阶段表现出更高的稳定性和准确性。实验结果表明，经过语义扩充和优化的问题能够显著提升模型对问题核心的把握能力，为后续的推理和答案生成奠定了坚实的基础。这种基于 **prompt** 工程的语义优化方法，不仅增强了大模型对复杂问题的理解能力，还为知识图谱问答任务提供了一种可扩展且高效的技术路径。

5 结论与展望

5.1 结论

本研究围绕知识图谱问答任务展开深入探索，针对现有大语言模型应用于该任务时的局限，提出了 Self-VEQA Agent 和 Iter-MKGRR 模型。主要内容如下：

(1) 对于 Self-VEQA Agent，首次在 KGQA 任务中引入多轮交互验证机制，构建了问答智能体与验证智能体协同运作的模式。该智能体无需微调，在 MetaQA 2Hop、MetaQA 3Hop 和 WebQSP 数据集的 KGQA 任务中，性能优于传统方法以及大多数基于大语言模型的方法。通过多轮自主验证，有效提升了大语言模型在 KGQA 任务中的推理能力，同时记忆机制的引入使模型具备动态进化能力，随着任务经验的积累，问答智能体的准确性逐步提高。实验表明，验证模块和记忆模块对提升问答性能都很重要。

(2) 对于 Iter-MKGRR 框架，本文提出了一种新的模型，将大语言模型引入 KGQA 任务，通过将外部知识图谱的检索结果与大模型的内部知识相结合，显著提升了复杂多跳问答任务的准确性。此外，本研究首次提出了一种针对多跳问题的新型处理方法，即在问题分解后，将每一步的答案与子问题融合，生成新的明确子问题，从而为大语言模型提供更连贯的信息，实验证明该方法有效提高了知识检索和推理的效率与准确性。同时，通过精心设计的 prompt 工程，本研究引导模型更准确地理解问题语义，增强了其对问题核心的把握能力。

5.2 展望

尽管本研究在知识图谱问答领域取得了一定的成果，但仍存在一些不足之处，未来可从以下几个方面进行改进和拓展：

对于 Self-VEQA Agent，虽然 Self-VEQA Agent 在 MetaQA 和 WebQSP 数据集上表现良好，但在更多类型和领域的数据集上的泛化能力还有待进一步验证和提升。未来可以尝试在不同领域、不同规模的数据集上进行实验，以增强模型的通用性。除此之外，目前问答智能体和验证智能体之间的交互轮数和推理链数量等参数是通过实验确定的，缺乏更智能的自适应调整策略。未来可以研究如何根据问题的复杂程度和数据集的特点，动态调整这些参数，以提高智能体的性能。

对于 Iter-MKGRR 框架，虽然 Iter-MKGRR 框架在处理复杂多跳问题上表现出色，但在大规模知识图谱和大量问题的情况下，模型的运行效率可能会受到影响。未来可以研究如何优化模型的算法和架构，提高知识检索和推理的效率。

参 考 文 献

- [1] Xu K, Lai Y, Feng Y, et al. Enhancing key-value memory neural networks for knowledge based question answering[C]. 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), Minneapolis, Minnesota, USA. 2019: 2937-2947.
- [2] Saxena A, Tripathi A, Talukdar P. Improving multi-hop question answering over knowledge graphs using knowledge base embeddings[C]. 58th Annual Meeting of Association for Computational Linguistics (ACL), Online. 2020: 4498-4507.
- [3] Zhao R, Zhao F, Wang L, et al. Kg-cot: Chain-of-thought prompting of large language models over knowledge graphs for knowledge-aware question answering[C]. Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI), Jeju, South Korea. 2024: 6642-6650.
- [4] Jiang J, Zhou K, Dong Z, et al. Structgpt: A general framework for large language model to reason over structured data[C]. 2023 Conference on Empirical Methods in Natural Language Processing, Singapore. 2023: 9237-9251.
- [5] Shah M, Tian J. Improving LLM-based KGQA for multi-hop Question Answering with implicit reasoning in few-shot examples[C]. 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM), Bangkok, Thailand. 2024: 125-135.
- [6] Luo L, Li Y F, Haffari G, et al. Reasoning on graphs: Faithful and interpretable large language model reasoning[C]. The Twelfth International Conference on Learning Representations, Vienna, Austria. 2024.
- [7] Cheng S, Zhuang Z, Xu Y, et al. Call Me When Necessary: LLMs can Efficiently and Faithfully Reason over Structured Environments[C]. Association for Computational Linguistics (ACL), Bangkok, Thailand. 2024: 4275-4295.
- [8] Yih W, Richardson M, Meek C, et al. The value of semantic parse labeling for knowledge base question answering[C]. 54th Annual Meeting of the Association for Computational Linguistics (ACL), Berlin, Germany. 2016: 201-206.
- [9] Arora S, Narayan A, Chen M F, et al. Ask me anything: A simple strategy for prompting language models[C]. The Eleventh International Conference on Learning Representations, Kigali, Rwanda. 2023.
- [10] Sun T X, Liu X Y, Qiu X P, et al. Paradigm shift in natural language processing[J]. Machine Intelligence Research, 2022, 19(3): 169-183.
- [11] Clusmann J, Kolbinger F R, Muti H S, et al. The future landscape of large language models in medicine[J]. Communications medicine, 2023, 3(1): 141.
- [12] Cheng Z, Xie T, Shi P, et al. Binding language models in symbolic languages[C]. The Eleventh International Conference on Learning Representations, Kigali, Rwanda. 2023.
- [13] Touvron H, Lavril T, Izacard G, et al. Llama: Open and efficient foundation language models[J]. ArXiv, 2023, abs/2302.13971.
- [14] Touvron H, Martin L, Stone K, et al. Llama 2: Open foundation and fine-tuned chat models[J]. ArXiv, 2023, abs/2307.09288.
- [15] Bai J, Bai S, Chu Y, et al. Qwen technical report[J]. ArXiv, 2023, abs/2309.16609.
- [16] Achiam J, Adler S, Agarwal S, et al. Gpt-4 technical report[J]. ArXiv, 2023, abs/2303.08774.

- [17] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in neural information processing systems, 2022, 35: 24824-24837.
- [18] Jie Z, Luong T Q, Zhang X, et al. Design of Chain-of-Thought in Math Problem Solving[J]. ArXiv, 2023, abs/2309.11054.
- [19] Zhao X, Li M, Lu W, et al. Enhancing zero-shot chain-of-thought reasoning in large language models through logic[C]. Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), Torino, Italia. 2024: 6144-6166.
- [20] Zhang Z, Zhang A, Li M, et al. Automatic chain of thought prompting in large language models[C]. The Eleventh International Conference on Learning Representations, Kigali, Rwanda. 2023.
- [21] Kojima T, Gu S S, Reid M, et al. Large language models are zero-shot reasoners[J]. Advances in neural information processing systems, 2022, 35: 22199-22213.
- [22] Fu Y, Peng H, Sabharwal A, et al. Complexity-based prompting for multi-step reasoning[C]. The Eleventh International Conference on Learning Representations, Kigali, Rwanda. 2023.
- [23] Yao S, Yu D, Zhao J, et al. Tree of thoughts: Deliberate problem solving with large language models[J]. Advances in Neural Information Processing Systems, 2024, 36: 11809-11822.
- [24] Besta M, Blach N, Kubicek A, et al. Graph of thoughts: Solving elaborate problems with large language models[C]. AAAI Conference on Artificial Intelligence (AAAI), Vancouver, Canada. 2024, 38(16): 17682-17690.
- [25] He H, Zhang H, Roth D. Rethinking with retrieval: Faithful large language model inference[J]. ArXiv, 2022, abs/2301.00303.
- [26] Wang K, Duan F, Wang S, et al. Knowledge-driven cot: Exploring faithful reasoning in llms for knowledge-intensive question answering[J]. ArXiv, 2023, abs/2308.13259.
- [27] Sun J, Xu C, Tang L, et al. Think-on-graph: Deep and responsible reasoning of large language model with knowledge graph[C]. The Twelfth International Conference on Learning Representations, Vienna, Austria. 2024.
- [28] Xiong G, Bao J, Zhao W. Interactive-kbqa: Multi-turn interactions for knowledge base question answering with large language models[C]. 62nd Annual Meeting of the Association for Computational Linguistics, Bangkok, Thailand. 2024: 10561–10582.
- [29] Sen P, Mavadia S, Saffari A. Knowledge graph-augmented language models for complex question answering[C]. 1st Workshop on Natural Language Reasoning and Structured Explanations (NLRSE), Toronto, Canada. 2023: 1-8.
- [30] Li X, Zhao R, Chia Y K, et al. Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources[C]. The Twelfth International Conference on Learning Representations, Vienna, Austria. 2024.
- [31] Pan S, Luo L, Wang Y, et al. Unifying large language models and knowledge graphs: A roadmap[J]. IEEE Transactions on Knowledge and Data Engineering, 2024, 36(7): 3580-3599.
- [32] Jiang J, Zhou K, Zhao W X, et al. Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph[C]. The Eleventh International Conference on Learning Representations, Kigali, Rwanda. 2023.

- [33] He G, Lan Y, Jiang J, et al. Improving multi-hop knowledge base question answering by learning intermediate supervision signals[C]. 14th ACM international conference on web search and data mining (WSDM), Online. 2021: 553-561.
- [34] Dong G, Zhao J, Hui T, et al. Revisit input perturbation problems for llms: A unified robustness evaluation framework for noisy slot filling task[C]. CCF International Conference on Natural Language Processing and Chinese Computing (NLPCC), Foshan, Guangdong, China. 2023: 682-694.
- [35] Buehler M J. Generative retrieval-augmented ontologic graph and multiagent strategies for interpretive large language model-based materials design[J]. ACS Engineering Au, 2024, 4(2): 241-277.
- [36] Guo T, Yang Q, Wang C, et al. Knowledgenavigator: Leveraging large language models for enhanced reasoning over knowledge graph[J]. Complex & Intelligent Systems, 2024, 10(5): 7063-7076.
- [37] Kim J, Kwon Y, Jo Y, et al. KG - GPT: A general framework for reasoning on knowledge graphs using large language models[C]. 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), Singapore. 2023: 9410 - 9421.
- [38] Wu Y, Hu N, Bi S, et al. Retrieve-rewrite-answer: A kg-to-text enhanced llms framework for knowledge graph question answering[J]. CoRR, 2023, abs/2309.11206.
- [39] Madani N, Srihari R K, Joseph K. Domain Specific Question Answering Over Knowledge Graphs Using Logical Programming and Large Language Models[J]. ArXiv, 2023, abs/2303.02206.
- [40] Yang L, Chen H, Li Z, et al. Give us the facts: Enhancing large language models with knowledge graphs for fact-aware language modeling[J]. IEEE Transactions on Knowledge and Data Engineering, 2024, 36(7): 3091-3110.
- [41] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.
- [42] Kenton J D M W C, Toutanova L K. Bert: Pre-training of deep bidirectional transformers for language understanding[C]. 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Minneapolis, Minnesota, USA. 2019, 1(2).
- [43] Tian L, Zhou X, Wu Y P, et al. Knowledge graph and knowledge reasoning: A systematic review[J]. Journal of Electronic Science and Technology, 2022, 20(2): 100159.
- [44] Chakraborty N, Lukovnikov D, Maheshwari G, et al. Introduction to neural network-based question answering over knowledge graphs[J]. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 2021, 11(3): e1389.
- [45] Choi H K, Lee S, Chu J, et al. NuTrea: neural tree search for context-guided multi-hop KGQA[J]. Advances in Neural Information Processing Systems, 2023, 36: 35954-35965.
- [46] Martino A, Iannelli M, Truong C. Knowledge injection to counter large language model (LLM) hallucination[C]. European Semantic Web Conference (ESWC), Hersonissos, Crete, Greece. 2023: 182-185.
- [47] Huang X, Zhang J, Li D, et al. Knowledge graph embedding based question answering[C]. Twelfth ACM International Conference on Web Search and Data Mining (WSDM), Melbourne, Australia. 2019: 105-113.
- [48] Yu D, Zhu C, Yang Y, et al. Jaket: Joint pre-training of knowledge graph and language understanding[C]. AAAI Conference on Artificial Intelligence (AAAI), Online. 2022, 36(10): 11630-11638.

- [49] Qiao Z, Li N, Huang C, et al. GraphLLM: A General Framework for Multi-hop Question Answering over Knowledge Graphs Using Large Language Models[C]. CCF International Conference on Natural Language Processing and Chinese Computing (NLPCC), Singapore. 2024: 136-148.
- [50] Liu J, Zhou T, Chen Y, et al. Enhancing Large Language Models with Pseudo-and Multisource-Knowledge Graphs for Open-ended Question Answering[J]. ArXiv, 2024, abs/2402.09911.
- [51] Taunk D, Khanna L, Kandru S V P K, et al. GrapeQA: Graph augmentation and pruning to enhance question-answering[C]. ACM Web Conference 2023(WWW), Austin, Texas, USA. 2023: 1138-1144.
- [52] Jiang J, Zhou K, Zhao W X, et al. ReasoningLM: Enabling structural subgraph reasoning in pre-trained language models for question answering over knowledge graph[C]. 2023 Conference on Empirical Methods in Natural Language Processing, Singapore. 2023: 3721-3735.
- [53] Baek J, Aji A F, Saffari A. Knowledge - augmented language model prompting for zero - shot knowledge graph question answering[C]. 1st Workshop on Natural Language Reasoning and Structured Explanations, NLRSE, Toronto, Canada. 2023: 78 - 106.
- [54] Feng Y, Chen X, Lin B Y, et al. Scalable multi-hop relational reasoning for knowledge-aware question answering[C]. 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Online. 2020: 1295-1309.
- [55] Dammu P P S, Naidu H, Dewan M, et al. ClaimVer: Explainable Claim-Level Verification and Evidence Attribution of Text Through Knowledge Graphs[C]. 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), Miami, Florida, USA. 2024:13613-13627.
- [56] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[J]. Advances in neural information processing systems, 2022, 35: 27730-27744.
- [57] Zhang J, Zhang X, Yu J, et al. Subgraph retrieval enhanced model for multi-hop knowledge base question answering[C]. 60th Annual Meeting of the Association for Computational Linguistics, Dublin, Ireland. 2022:5773-5784.
- [58] Liu W, Zhou P, Zhao Z, et al. K-bert: Enabling language representation with knowledge graph[C]. AAAI Conference on Artificial Intelligence (AAAI), New York, USA. 2020, 34(03): 2901-2908.
- [59] Liu X, Yu H, Zhang H, et al. Agentbench: Evaluating llms as agents[C]. The Twelfth International Conference on Learning Representations, Vienna, Austria. 2024.
- [60] Huang R, Wei W, Qu X, et al. Joint Multi-Facts Reasoning Network for Complex Temporal Question Answering Over Knowledge Graph[C]. 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Seoul, South Korea. IEEE, 2024: 10331-10335.
- [61] Xu Z, Cruz M J, Guevara M, et al. Retrieval-augmented generation with knowledge graphs for customer service question answering[C]. 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), Washington, D.C., USA. 2024: 2905-2909.
- [62] Park J S, O'Brien J, Cai C J, et al. Generative agents: Interactive simulacra of human behavior[C]. 36th Annual ACM Symposium on User Interface Software and Technology (UIST), San Francisco, California, USA. 2023: 1-22.
- [63] Zhang Y, Dai H, Kozareva Z, et al. Variational reasoning for question answering with knowledge graph[C]. AAAI Conference on Artificial Intelligence (AAAI), New Orleans, Louisiana, USA. 2018, 32(1).
- [64] OpenAI. Gpt - 4o system card [EB/OL]. 2024, <https://www.openai.com/research/gpt-4o>.

- [65] Anil R, Dai A M, Firat O, et al. Palm 2 technical report[J]. ArXiv, 2023, abs/2305.10403.
- [66] Bollacker K, Evans C, Paritosh P, et al. Freebase: a collaboratively created graph database for structuring human knowledge[C]. 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD), Vancouver, British Columbia, Canada. 2008: 1247-1250.
- [67] Kosten C, Cudré-Mauroux P, Stockinger K. Spider4SPARQL: A Complex Benchmark for Evaluating Knowledge Graph Question Answering Systems[C]. 2023 IEEE International Conference on Big Data, Osaka, Japan. 2023: 5272-5281.
- [68] Shi J, Cao S, Hou L, et al. Transfernet: An effective and transparent framework for multi-hop question answering over relation graph[C]. 2021 Conference on Empirical Methods in Natural Language Processing, Online and Punta Cana, Dominican Republic. 2021: 4149-4158.
- [69] Shirafuji A, Oda Y, Suzuki J, et al. Refactoring programs using large language models with few-shot examples[C]. 2023 30th Asia-Pacific Software Engineering Conference (APSEC), Seoul, South Korea, 2023: 151-160.
- [70] Hou K, Li J, Liu Y, et al. KG-EGV: A Framework for Question Answering with Integrated Knowledge Graphs and Large Language Models[J]. Electronics, 2024, 13(23): 4835.
- [71] Guan X, Liu Y, Lin H, et al. Mitigating large language model hallucinations via autonomous knowledge graph-based retrofitting[C]. AAAI Conference on Artificial Intelligence (AAAI), Vancouver, Canada. 2024, 38(16): 18126-18134.
- [72] Chen X, Dong L, Han M. A Comprehensive Evaluation Method for KG-Augmented Large Language Models[C]. International Conference on Smart Grid Inspired Future Technologies, Hong Kong, China. 2024: 112-121.
- [73] Wijaya O C, Purwarianti A. An Interactive Question-Answering System Using Large Language Model and Retrieval-Augmented Generation in an Intelligent Tutoring System on the Programming Domain[C]. 2024 11th International Conference on Advanced Informatics: Concept, Theory and Application (ICAICTA), Singapore. 2024: 1-6.